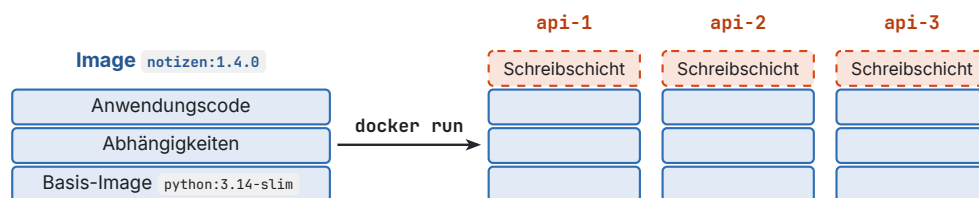


Docker von Grund auf

Container per Kommandozeile am Mac, Dockerfiles, Compose, Betrieb auf dem Server und CI/CD mit Gitea



Inhalt

Bevor es losgeht	4	6.4 Netzwerktreiber	22
Aufbau	4	6.5 Netzwerke untersuchen	22
Konventionen	4	7 Docker Compose	23
TEIL I · GRUNDLAGEN		7.1 Die erste <code>compose.yaml</code>	23
1 Das mentale Modell	6	7.2 Die Befehle	24
1.1 Das Problem, das Docker löst	6	7.3 Variablen: <code>.env</code> und <code>env_file</code>	24
1.2 Container sind keine virtuellen Maschinen	6	7.4 Startreihenfolge, Healthchecks und Migrationen	25
1.3 Was ein Container technisch ist	6	7.5 Entwickeln mit Compose Watch	25
1.4 Image und Container	7	7.6 Profile: optionale Dienste	26
1.5 Registry, Namen, Tags und Digests	7	7.7 Entwicklung und Produktion trennen	26
1.6 Die Architektur hinter dem <code>docker</code> -Befehl	8	8 Gute Images bauen	27
2 Docker am Mac ohne GUI: Colima	9	8.1 Das Basis-Image wählen	27
2.1 Die Optionen im Vergleich	9	8.2 Multi-Stage-Builds	27
2.2 Installation	9	8.3 Das Produktions-Dockerfile	27
2.3 Colima im Alltag	10	8.4 Versionen festlegen	28
2.4 Kontexte: zwischen Colima und Docker Desktop wechseln	11	8.5 Metadaten	29
2.5 Umstieg von Docker Desktop	11	9 ARM und x86: Builds für mehrere Plattformen	30
3 Container benutzen	12	9.1 Plattform gezielt wählen	30
3.1 <code>docker run</code> im Detail	12	9.2 Ein Name, mehrere Plattformen	30
3.2 Der Lebenszyklus eines Containers	12	9.3 Empfehlung für die Praxis	30
3.3 Die wichtigsten Befehle	13	10 Fehlersuche	32
3.4 Umgebungsvariablen und Konfiguration	13	10.1 Das Vorgehen	32
3.5 Neustart-Richtlinien	14	10.2 Exit-Codes lesen	32
3.6 Aufräumen	14	10.3 Häufige Probleme	33
4 Images und das Dockerfile	15	TEIL III · BETRIEB	
4.1 Images verwalten	15	11 Registries: Images speichern und verteilen	34
4.2 Das erste Dockerfile	15	11.1 Die Gitea-Registry	34
4.3 Die Anweisungen	16	11.2 Eine Tag-Strategie	34
4.4 Build-Kontext und <code>.dockerignore</code>	16	11.3 Docker Hub	35
4.5 Der Schicht-Cache	16	12 Sicherheit	36
4.6 <code>CMD</code> , <code>ENTRYPOINT</code> und die Signale	17	12.1 Der Docker-Socket ist Root	36
TEIL II · ZUSAMMENSPIEL		12.2 Minimale Rechte im Container	36
5 Daten: Volumes und Bind Mounts	18	12.3 Geheimnisse	37
5.1 Drei Arten, Daten einzubinden	18	12.4 Images auf Schwachstellen prüfen	37
5.2 Volumes	18	12.5 Herkunft und Integrität von Images	38
5.3 Bind Mounts in der Entwicklung	19	12.6 Checkliste	38
5.4 Sichern und Wiederherstellen	19	13 Betrieb auf dem Server	39
6 Netzwerke	21	13.1 Docker Engine installieren	39
6.1 Eigene Netzwerke und Namensauflösung	21	13.2 Aufbau auf dem Server	40
6.2 Ports veröffentlichen	21	13.3 Reverse Proxy mit automatischem HTTPS	40
6.3 Vom Container zum Host	22	13.4 Die Produktions-Compose-Datei	40

13.5 Updates und Rollbacks	41
13.6 Backups	42
13.7 Beobachten	43
14 Bauen und Ausliefern mit Gitea Actions	44
14.1 Vertrauenswürdige Werkzeuge im Runner	44
14.2 Tests gegen eine echte Datenbank	44
14.3 Image bauen, attestieren und als Kandidat veröffentlichen	45
14.4 Das Deployment-Skript für Compose	46
15 Compose, Swarm oder Kubernetes?	48
15.1 Ist Docker Swarm tot?	48
15.2 Die Optionen im Vergleich	48
15.3 Empfehlung	48
 ANHANG	
A Befehlsübersicht	50
B Glossar	51
Literaturverzeichnis	53

Bevor es losgeht

Dieses Handbuch ist das Gegenstück zu „Git von Grund auf“ und folgt demselben Prinzip: erst ein belastbares mentales Modell, dann die Befehle, dann der Betrieb. Wer Docker bisher vor allem über kopierte `docker run`-Zeilen und fremde `compose.yaml`-Dateien kennt, stößt früher oder später auf Fragen wie: Warum sind meine Daten nach einem Update weg? Warum startet das Image am Server nicht, obwohl es am Mac lief? Warum erreicht der Container die Datenbank nicht? Die Antworten ergeben sich fast immer aus wenigen Grundbegriffen.

Aufbau

- **Teil I, Grundlagen:** was ein Container technisch ist, Docker am Mac rein über die Kommandozeile (Colima statt Docker Desktop), Container bedienen und das erste Dockerfile.
- **Teil II, Zusammenspiel:** Daten und Volumes, Netzwerke, Docker Compose, schlanke und sichere Images, Builds für ARM und x86, systematische Fehlersuche.
- **Teil III, Betrieb:** Registry in Gitea, Sicherheit, Betrieb auf einem Linux-Server mit Reverse Proxy, Updates und Backups, Builds und Deployment mit Gitea Actions sowie eine Einordnung von Swarm und Kubernetes.
- **Anhang:** Befehlsübersicht und Glossar.

Für Gitea, Pull Requests, Runner, Secrets und SSH-Deployment verweist dieses Buch auf das Git-Handbuch (dort Kapitel 12 bis 15) [1], statt alles zu wiederholen.

Konventionen

Befehle stehen in grauen Kästen, Ausgaben in gestrichelten, Dateiinhalte tragen den Dateinamen als Reiter. In den Grafiken haben Farben eine feste Bedeutung:

 Image / Schicht	 Container	 Volume / Daten
 Netzwerk	 Registry / Gitea	 Host / VM

Als durchgängiges Beispiel dient das Projekt `notizen`: eine kleine REST-API mit FastAPI [2], die Notizen in PostgreSQL speichert. Abhängigkeiten verwaltet `uv` [3]. Es ist bewusst klein, enthält aber alles, was in echten Projekten vorkommt: eine Anwendung, eine Datenbank mit persistenten Daten, Konfiguration über Umgebungsvariablen, Geheimnisse, Migrationen und Tests.

Projektstruktur

```
notizen/
├── app/
│   ├── __init__.py
│   ├── main.py          # FastAPI-Anwendung
│   └── db.py            # Datenbankverbindung
├── tests/
├── pyproject.toml       # Abhängigkeiten
├── uv.lock              # exakt festgelegte Versionen
├── Dockerfile
├── .dockerignore
├── compose.yaml         # Entwicklung: api + db
└── compose.prod.yaml    # Produktion (liegt auf dem Server als compose.yaml)
```

Server heißen wie im Git-Handbuch `gitea.example.com` (Gitea mit Container-Registry) und `app.example.com` (Zielserver). Getestete Basis dieses Handbuchs sind Docker Engine 29.8 und Docker Compose 5.5 im September 2026 [4], [5]. Die gezeigten Grundfunktionen funktionieren auch mit aktuellen Compose-v2-Versionen; neuere Funktionen sind im Text ausdrücklich gekennzeichnet. Der Befehl heißt weiterhin

`docker compose` (mit Leerzeichen). Das alte, separate Programm `docker-compose` mit Bindestrich ist veraltet [6].

1 Das mentale Modell

Docker wirkt wie eine leichte virtuelle Maschine, ist aber etwas grundlegend anderes. Viele Stolpersteine, etwa verschwundene Daten, unerreichbare Ports oder Images, die auf dem Server nicht starten, lösen sich auf, sobald man das Modell dahinter kennt. Es besteht aus vier Begriffen: **Image**, **Container**, **Registry** und dem **Docker-Daemon**.

1.1 Das Problem, das Docker löst

Eine Anwendung braucht mehr als ihren Quellcode: eine bestimmte Python-Version, Systembibliotheken, Python-Pakete in exakten Versionen, Konfiguration. Auf dem eigenen Mac ist all das irgendwann über Monate gewachsen, auf dem Server anders, beim Kollegen wieder anders. Das Ergebnis ist der Klassiker „läuft bei mir“.

Docker verpackt die Anwendung mit ihren Benutzerland-Bibliotheken und Werkzeugen in ein Image [7], [8]. Kernel, CPU-Architektur und einige Host-Funktionen kommen weiterhin vom Zielsystem [9], [10]. Innerhalb einer unterstützten Plattform läuft dasselbe Image deshalb weitgehend gleich: am Mac, im CI-Runner und auf dem Produktionsserver [11]. Die Laufzeitumgebung wird damit zu einem versionierbaren Artefakt statt zu schwer nachvollziehbarem Zustand eines Rechners.

1.2 Container sind keine virtuellen Maschinen

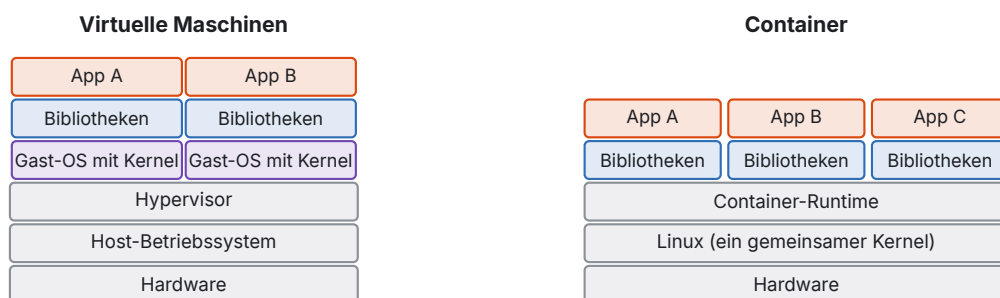


Abb. 1: Eine VM bringt ein komplettes Betriebssystem mit eigenem Kernel mit. Container teilen sich den Kernel des Hosts.

Eine virtuelle Maschine emuliert einen ganzen Rechner, inklusive eigenem Betriebssystem-Kernel [9], [11]. Sie bootet dafür ein eigenes Betriebssystem und reserviert Arbeitsspeicher für ein komplettes System. Ein Container dagegen ist **ein ganz normaler Prozess auf dem Linux-Host**, den der Kernel so abschottet, dass er glaubt, allein zu sein [9], [12]. Er startet deshalb meist deutlich schneller und braucht kaum mehr Speicher als die Anwendung selbst.

1.3 Was ein Container technisch ist

Drei Kernel-Mechanismen machen aus einem Prozess einen Container:

Mechanismus	Wirkung
Namespaces	Eigene Sicht auf das System: eigene Prozessliste (der Hauptprozess hat PID 1), eigenes Netzwerk mit eigener IP, eigene Mount-Punkte, eigener Hostname, optional eigene Benutzer-IDs [12], [13].
cgroups	Begrenzung und Messung von Ressourcen: CPU, Arbeitsspeicher, Anzahl der Prozesse, Ein-/Ausgabe [14].
Eigenes Dateisystem	Der Prozess sieht als Wurzelverzeichnis <code>/</code> nicht das Dateisystem des Hosts, sondern den Inhalt des Images [15].

Daraus folgen drei Eigenschaften, die man im Alltag ständig braucht:

1. **Ein Container lebt genau so lange wie sein Hauptprozess.** Beendet sich der Prozess (weil das Programm fertig ist oder abstürzt), ist der Container beendet [13]. Ein Container ohne dauerhaft laufenden Prozess „startet und ist sofort wieder weg“.
2. **Container brauchen einen Linux-Kernel.** Auf macOS gibt es keinen, deshalb läuft Docker am Mac immer in einer kleinen Linux-VM (Kapitel 2) [16].
3. **Die Isolation ist gut, aber nicht so stark wie bei einer VM.** Alle Container teilen sich einen Kernel [10]. Wer im Container Root-Rechte hat und aus ihm ausbricht, steht auf dem Host [17] (Kapitel 12).

1.4 Image und Container

Das wichtigste Begriffspaar: Ein **Image** ist eine unveränderliche Vorlage, ein **Container** ist eine laufende (oder gestoppte) Instanz davon [7], [9]. Die Beziehung entspricht der zwischen Klasse und Objekt: Aus einem Image lassen sich beliebig viele Container starten.

Ein Image besteht aus übereinander liegenden, schreibgeschützten **Schichten** (*layers*). Jede Anweisung im Dockerfile, die Dateien verändert, erzeugt eine Schicht. Startet man einen Container, legt Docker darüber eine dünne, beschreibbare Schicht, die nur diesem Container gehört [15].

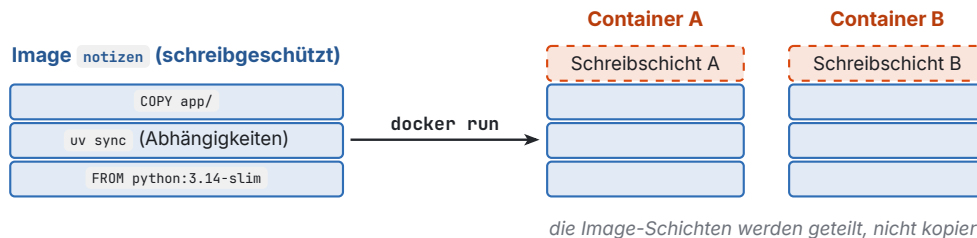


Abb. 2: Beide Container teilen sich die Schichten des Images. Jeder hat seine eigene Schreibschrift.

Ändert ein Container eine Datei aus dem Image, kopiert Docker sie zuerst in seine Schreibschrift (*copy-on-write*) [15]. Das Image selbst bleibt unverändert. Daraus folgt die wichtigste Regel für den Umgang mit Daten:

MERKE

Die Schreibschrift wird zusammen mit dem Container gelöscht [15]. Alles, was einen Neustart mit neuem Image überleben soll (Datenbankdateien, Uploads), gehört in ein **Volume** (Kapitel 5). Container sind Wegwerfware, Daten nicht.

1.5 Registry, Namen, Tags und Digests

Images werden in einer **Registry** gespeichert und von dort geladen: Docker Hub ist die Standard-Registry [8], Gitea bringt eine eigene mit (Kapitel 11). Ein vollständiger Image-Name besteht aus mehreren Teilen:



Abb. 3: Aufbau eines Image-Namens. Fehlt die Registry, ist Docker Hub gemeint.

Kurzformen werden ergänzt: `python:3.14-slim` bedeutet vollständig `docker.io/library/python:3.14-slim`. Fehlt der Tag, nimmt Docker `latest` [18].

Wer das Git-Handbuch kennt, findet hier eine vertraute Unterscheidung wieder: Ein **Tag** ist ein beweglicher Name wie ein Branch. `python:3.14-slim` zeigt heute auf ein anderes Image als in drei Monaten, weil Sicherheitsupdates eingespielt werden [19]. Ein **Digest** (`@sha256:...`) ist dagegen die Prüfsumme des Inhalts, unveränderlich wie ein Commit-Hash [18]. Wer exakt reproduzierbar bauen will, pinnt auf den Digest und aktualisiert ihn kontrolliert [19].

ACHTUNG

`latest` bedeutet nicht „neueste Version“, sondern nur „der Tag, der verwendet wird, wenn keiner angegeben ist“ [18]. Was sich dahinter verbirgt, entscheidet der Herausgeber. In Produktion und in Dockerfiles immer eine konkrete Version angeben.

1.6 Die Architektur hinter dem `docker`-Befehl

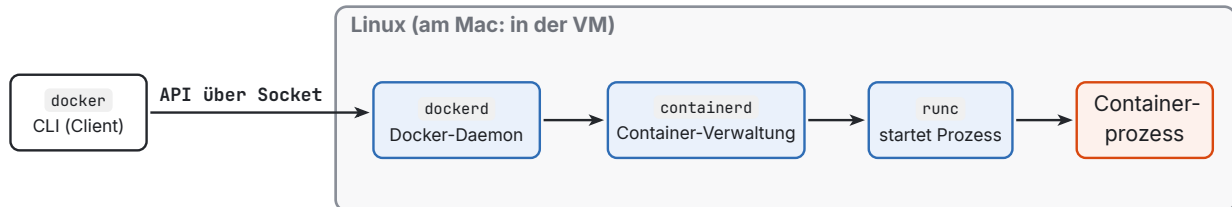


Abb. 4: Der `docker`-Befehl ist nur ein Client. Die eigentliche Arbeit erledigt der Daemon.

Das Programm `docker`, das du im Terminal aufrufst, ist nur ein **Client**. Es schickt jeden Befehl über eine Programmierschnittstelle an den **Docker-Daemon** `dockerd`, der Images verwaltet, Netzwerke und Volumes anlegt und Container über `containerd` und `runc` startet [8]. Die Verbindung läuft über einen Unix-Socket (auf Linux `/var/run/docker.sock`) [20].

Das erklärt zwei Dinge. Erstens kann der Client auf einem anderen Rechner laufen als der Daemon: am Mac läuft der Client unter macOS, der Daemon in der Linux-VM. Zweitens ist **Zugriff auf den Socket gleichbedeutend mit Root-Rechten** auf dem Rechner des Daemons, denn wer Container starten darf, kann beliebige Verzeichnisse des Hosts einbinden [17] (Kapitel 12).

MERKE

Das Modell in fünf Sätzen: **(1)** Ein Container ist ein abgeschotteter Linux-Prozess. **(2)** Er entsteht aus einem unveränderlichen, geschichteten Image. **(3)** Seine eigenen Änderungen landen in einer Schreibschicht, die mit ihm verschwindet. **(4)** Images liegen in Registries und werden über Name, Tag oder Digest angesprochen. **(5)** Der `docker`-Befehl ist nur ein Client für den Daemon.

2 Docker am Mac ohne GUI: Colima

Weil Container einen Linux-Kernel brauchen, läuft Docker am Mac immer in einer Linux-VM. Die Frage ist nur, wer diese VM verwaltet. Docker Desktop bringt dafür eine grafische Anwendung mit. Wer ohnehin nur im Terminal arbeitet, kommt mit **Colima** aus: ein reines Kommandozeilenwerkzeug, das die VM startet und den normalen `docker`-Befehl mit ihr verbindet [21].

2.1 Die Optionen im Vergleich

Werkzeug	Oberfläche	Einordnung
Docker Desktop	GUI + CLI	Offizielle Lösung mit vielen Extras. Kostenlos für Privatanutzung, Ausbildung, nicht-kommerzielle Open-Source-Projekte und kleine Unternehmen (unter 250 Beschäftigte und unter 10 Mio. USD Umsatz), darüber kostenpflichtig [22]. Läuft als Hintergrund-App.
Colima	nur CLI	Open Source (MIT), basiert auf Lima [21]. Startet eine schlanke VM mit Docker-Daemon, danach funktioniert der normale <code>docker</code> -Befehl [21]. Keine Lizenzfragen, kein GUI-Prozess.
OrbStack	GUI + CLI	Laut Hersteller sehr schnell und sparsam, aber kommerzielles Produkt (kostenlos nur für private, nicht-kommerzielle Nutzung, beruflich lizenzpflichtig) [23].
Podman	CLI, optional GUI	Alternative Container-Engine ohne Daemon, weitgehend Docker-kompatibel, mit gelegentlichen Unterschieden bei Compose [24].

Für eine reine Terminal-Arbeitsweise ist Colima die naheliegende Wahl. Die Docker-CLI und die Beispiele dieses Buchs funktionieren damit weitgehend wie unter Docker Desktop. Details der VM-Integration, Netzwerke, Dateifreigaben und mitgelieferte Zusatzfunktionen unterscheiden sich jedoch.

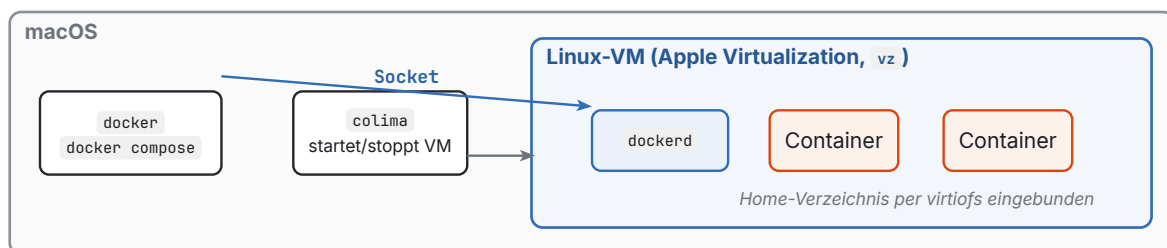


Abb. 5: Colima verwaltet nur die VM. Der `docker`-Befehl spricht direkt mit dem Daemon darin.

2.2 Installation

```
# 1. Rosetta 2 installieren (für x86-Images, einmalig, nur bis macOS 26 nötig, VOR dem ersten
Start von Colima)
softwareupdate --install-rosetta --agree-to-license

# 2. Colima, Docker-CLI und Plugins über Homebrew
brew install colima docker docker-compose docker-buildx \
  docker-credential-helper
```

Compose und Buildx sind bei der Homebrew-Installation **Plugins** der Docker-CLI [25]. Damit `docker compose` und `docker buildx` sie finden und Registry-Zugangsdaten im macOS-Schlüsselbund statt im Klartext landen [26], braucht die Docker-Konfiguration zwei Einträge:

```
~/ .docker/config.json
```

```
{
  "credsStore": "osxkeychain",
  "cliPluginsExtraDirs": ["/opt/homebrew/lib/docker/cli-plugins"]
}
```

Dann die VM erstmals starten, passend für Apple Silicon:

```
colima start --vm-type vz --vz-rosetta --mount-type virtiofs --cpu 4 --memory 6 --disk 60
```

Option	Bedeutung
<code>--vm-type vz</code>	Apples eigenes Virtualisierungs-Framework statt QEMU: schneller [27].
<code>--vz-rosetta</code>	x86-Programme in der ARM-VM über Rosetta ausführen. Nötig, um <code>linux/amd64</code> -Images zu testen (Kapitel 9). Ab macOS 27 ist die Übersetzung im System enthalten, eine Rosetta-Installation entfällt dann [28].
<code>--mount-type virtiofs</code>	Schnellste Art, Mac-Verzeichnisse in Container einzubinden [27].
<code>--cpu</code> , <code>--memory</code> , <code>--disk</code>	Ressourcen der VM (GB). Bei 16 bis 18 GB RAM im Mac sind 6 GB für die VM ein guter Start. Die Disk lässt sich später vergrößern, aber nicht verkleinern [27].

Prüfen, ob alles funktioniert:

```
docker context ls                # "colima" sollte mit * markiert sein
docker version                  # Client (macOS) und Server (Linux)
docker compose version
docker run --rm hello-world
docker run --rm --platform linux/amd64 alpine uname -m # zeigt x86_64 dank Rosetta
```

2.3 Colima im Alltag

Befehl	Wirkung
<code>colima start</code>	VM mit der gespeicherten Konfiguration starten
<code>colima stop</code>	VM herunterfahren (Container, Images und Volumes bleiben erhalten)
<code>colima status</code>	Zustand, Architektur, Laufzeit, Mount-Typ
<code>colima list</code>	alle Colima-Profile mit CPU, RAM und Disk
<code>colima start --edit</code>	Konfiguration im Editor ändern, danach Neustart der VM
<code>colima ssh</code>	Shell in der Linux-VM (selten nötig)
<code>colima delete</code>	VM löschen. Images und Volumes bleiben auf einer separaten Daten-Disk erhalten, <code>colima delete --data</code> löscht auch diese [29], [30]
<code>brew services start colima</code>	Colima bei jeder Anmeldung automatisch starten

Die Konfiguration liegt in `~/ .colima/default/colima.yaml`. Dort lassen sich die Startoptionen dauerhaft festlegen, sodass ein schlichtes `colima start` genügt [27]. Mehrere unabhängige VMs sind über Profile möglich (`colima start --profile test`) [31], was sich etwa zum Ausprobieren einer anderen Docker-Version eignet.

SPEZIELL AM MAC

Eingebundene Verzeichnisse: Standardmäßig ist nur dein Home-Verzeichnis in der VM sichtbar. Projekte außerhalb davon, etwa auf einem externen Laufwerk, müssen in `colima.yaml` unter `mounts:` eingetragen werden, sonst sieht der Container einen leeren Ordner [30].

Andere Werkzeuge: Programme, die den Docker-Socket direkt suchen (manche IDE-Plugins, Testcontainers), finden ihn nicht am Standardort. Falls ein Werkzeug keine Docker Contexts unterstützt, kann man für dessen einzelnen Aufruf `DOCKER_HOST="unix://$HOME/.colima/default/docker.sock"` `werkzeug ...` setzen [30]. Ein globaler Export in der `~/.zshrc` ist keine gute Lösung: `DOCKER_HOST` übersteuert den aktiven Context und macht `docker context use ...` scheinbar wirkungslos [32].

Speicherplatz: Alle Images und Volumes liegen in der Disk-Datei der VM. `docker system df` zeigt, wie viel davon belegt ist (Kapitel 10) [33].

2.4 Kontexte: zwischen Colima und Docker Desktop wechseln

Die CLI kann mit mehreren Daemons sprechen. Jede Verbindung ist ein **Kontext**. Colima legt beim Start den Kontext `colima` an [31], Docker Desktop den Kontext `desktop-linux` [34]. Der aktive Kontext entscheidet, wohin `docker` seine Befehle schickt [32]:

```
docker context ls          # alle Kontexte, der aktive ist mit * markiert
docker context use colima  # auf Colima umschalten
docker context use desktop-linux # zurück zu Docker Desktop
```

Kontexte können auch auf entfernte Rechner zeigen, etwa den Server per SSH (`docker context create server --docker "host=ssh://deploy@app.example.com"`). Das ist bequem, aber mit Vorsicht zu genießen: Mit dem falschen aktiven Kontext landet ein `docker compose down -v` auf dem Server statt am Mac. Wer so arbeitet, sollte sich den aktiven Kontext im Prompt anzeigen lassen oder den Kontext pro Befehl angeben (`docker --context server ps`).

2.5 Umstieg von Docker Desktop

1. Docker Desktop beenden und in dessen Einstellungen den Autostart abschalten.
2. Prüfen, welche `docker`-CLI verwendet wird: `which -a docker`. Docker Desktop legt Links auf seine Programme standardmäßig unter `~/.docker/bin` an (je nach Einstellung auch unter `/usr/local/bin`) [34]. Die Homebrew-Version liegt unter `/opt/homebrew/bin/docker` und sollte zuerst gefunden werden.
3. Images und Volumes liegen in der VM von Docker Desktop und kommen nicht automatisch mit. Images baut oder lädt man einfach neu. Wichtige Volume-Daten (etwa eine lokale Entwicklungsdatenbank) sichert man vorher mit einem Dump (Kapitel 5).
4. In `~/.docker/config.json` einen eventuell von Docker Desktop gesetzten `credsStore: "desktop"` durch `osxkeychain` ersetzen [26].
5. Docker Desktop kann parallel installiert bleiben, wenn es später doch einmal gebraucht wird. Der aktive Kontext entscheidet [32].

TIPP

Die Tab-Vervollständigung für `docker` und `docker compose` liefert das Homebrew-Paket `docker` bereits mit, aktiviert wird sie mit `compinit` in der `~/.zshrc` (siehe Git-Handbuch, Kapitel 2). Danach vervollständigt die Shell auch Container- und Image-Namen.

3 Container benutzen

Bevor es um eigene Images geht, lohnt es sich, mit fertigen Images vertraut zu werden: Container starten, beobachten, in sie hineinschauen und wieder aufräumen. Alle Beispiele nutzen offizielle Images von Docker Hub.

3.1 docker run im Detail

`docker run` lädt das Image (falls nicht vorhanden), legt einen Container an und startet ihn. Die Optionen stehen **vor** dem Image-Namen, alles **nach** dem Image-Namen ist der Befehl, der im Container ausgeführt wird [35].

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=geheim \
  -p 127.0.0.1:5432:5432 \
  -v pgdaten:/var/lib/postgresql \
  --restart unless-stopped \
  postgres:18
```

Option	Bedeutung
<code>-d</code>	im Hintergrund starten (<i>detached</i>), statt die Ausgabe im Terminal zu zeigen
<code>--name db</code>	fester Name. Ohne ihn vergibt Docker Zufallsnamen wie <code>eager_turing</code> .
<code>-e NAME=wert</code>	Umgebungsvariable setzen, der übliche Weg, Container zu konfigurieren
<code>-p 127.0.0.1:5432:5432</code>	Port veröffentlichen: <i>Host-Adresse:Host-Port:Container-Port</i> (Kapitel 6)
<code>-v pgdaten:/var/lib/postgresql</code>	Volume <code>pgdaten</code> an diesen Pfad im Container einbinden [36] (Kapitel 5)
<code>--restart unless-stopped</code>	nach Absturz oder Neustart des Daemons automatisch wieder starten [37]
<code>--rm</code>	Container nach dem Beenden automatisch löschen, ideal für Einmal-Aufgaben
<code>-it</code>	interaktiv mit Terminal, z.B. für eine Shell: <code>docker run --rm -it python:3.14-slim bash</code>

3.2 Der Lebenszyklus eines Containers

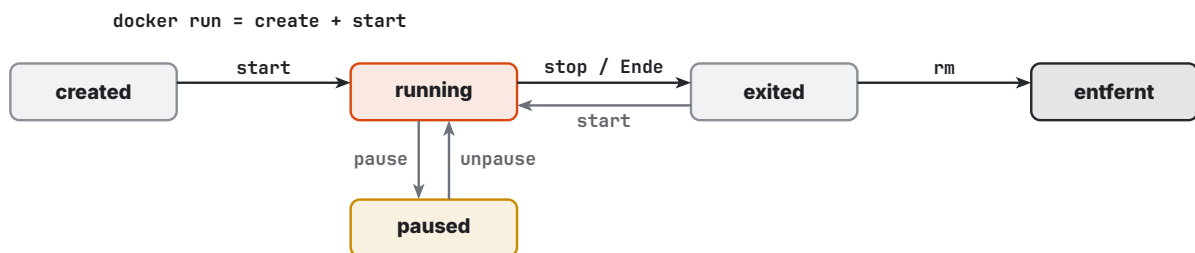


Abb. 6: Zustände eines Containers. Ein gestoppter Container existiert weiter, bis er entfernt wird.

Ein gestoppter Container ist nicht weg: Er behält seine Schreibschrift und seine Konfiguration und kann mit `docker start` fortgesetzt werden. Erst `docker rm` entfernt ihn. Deshalb sammeln sich ohne `--rm` mit der Zeit viele gestoppte Container an [35], [38].

3.3 Die wichtigsten Befehle

Befehl	Wirkung
<code>docker ps</code>	laufende Container anzeigen (<code>-a</code> zusätzlich gestoppte)
<code>docker logs -f db</code>	Ausgaben des Hauptprozesses fortlaufend anzeigen (<code>--tail 100</code> , <code>--since 10m</code>)
<code>docker exec -it db bash</code>	Befehl in einem laufenden Container ausführen, hier eine Shell [39]
<code>docker stop db</code> / <code>start</code> / <code>restart</code>	anhalten (erst SIGTERM, nach 10 s SIGKILL), fortsetzen, neu starten [40]
<code>docker rm db</code>	gestoppten Container entfernen (<code>-f</code> stoppt vorher)
<code>docker inspect db</code>	vollständige Konfiguration als JSON: IP, Mounts, Umgebung, Status [41]
<code>docker stats</code>	CPU-, Speicher- und Netzwerkverbrauch live
<code>docker top db</code>	Prozesse im Container
<code>docker cp db:/pfad/datei .</code>	Dateien zwischen Container und Mac kopieren
<code>docker port db</code>	veröffentlichte Ports anzeigen

Viele Namen sind abkürzbar: Statt des Namens funktioniert auch der Anfang der Container-ID (`docker logs 3f2a`). Die Formatierung von `docker ps` lässt sich anpassen, was für eine kompakte Übersicht praktisch ist:

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
```

run oder exec ?

Ein häufiges Missverständnis: `docker run` startet immer einen **neuen** Container. Wer in einem bereits laufenden Container etwas nachsehen will, braucht `docker exec`. Beispiel mit der Datenbank von oben:

```
docker exec -it db psql -U postgres # psql im laufenden DB-Container
```

```
psql (18.0 (Debian 18.0-1.pgdg13+1))
Type "help" for help.

postgres=#
```

`docker exec` funktioniert nur, solange der Container läuft, und die Shell muss im Image vorhanden sein [39]. Schlanke Images enthalten oft kein `bash`, dann hilft `sh`. Minimale Images (*distroless*) enthalten gar keine Shell [42] (Kapitel 8 und 10).

3.4 Umgebungsvariablen und Konfiguration

Container werden fast immer über Umgebungsvariablen konfiguriert. Welche ein Image versteht, steht in seiner Dokumentation auf Docker Hub [43]. Bei vielen Variablen ist eine Datei übersichtlicher [35]:

```
.env.db
```

```
POSTGRES_USER=notizen
POSTGRES_PASSWORD=entwicklung
POSTGRES_DB=notizen
```

```
docker run -d --name db --env-file .env.db -v pgdaten:/var/lib/postgresql postgres:18
```

ACHTUNG

Umgebungsvariablen sind für jeden sichtbar, der `docker inspect` ausführen darf [41], und landen leicht in Logs. Für Passwörter in Produktion gibt es bessere Wege (Kapitel 12). Dateien wie `.env.db` gehören in die `.gitignore`.

3.5 Neustart-Richtlinien

--restart	Verhalten
no	nie automatisch neu starten (Standard)
on-failure[:5]	nur bei Exit-Code ungleich 0, optional höchstens fünfmal
unless-stopped	immer, außer der Container wurde bewusst gestoppt. Empfehlung dieses Buchs für Dienste.
always	immer, auch nach einem bewussten Stopp, sobald der Daemon neu startet [37]

3.6 Aufräumen

Images, gestoppte Container, ungenutzte Netzwerke und der Build-Cache belegen mit der Zeit viel Platz [38]:

```
docker system df           # Übersicht: wie viel belegt was?
docker container prune     # alle gestoppten Container löschen
docker image prune         # "dangling" Images löschen (ohne Tag)
docker image prune -a      # alle Images löschen, die kein Container nutzt
docker builder prune       # Build-Cache leeren
docker system prune        # gestoppte Container, ungenutzte Netze, Images ohne Tag,
Build-Cache
```

ACHTUNG

`docker volume prune` und `docker system prune --volumes` entfernen standardmäßig ungenutzte **anonyme** Volumes [44], [45]. Benannte Volumes werden erst mit `docker volume prune -a` entfernt; `docker compose down -v` entfernt die im Projekt verwendeten benannten und anonymen Volumes [46]. Vor jedem Löschen mit `docker volume ls` beziehungsweise `docker compose config --volumes` prüfen, was betroffen ist.

4 Images und das Dockerfile

Ein eigenes Image beschreibt man in einer Textdatei namens `Dockerfile`: eine Abfolge von Anweisungen, die ausgehend von einem Basis-Image Schicht für Schicht die Laufzeitumgebung aufbaut. In diesem Kapitel entsteht ein erstes, funktionierendes Image für das Beispielprojekt. Kapitel 8 macht daraus später ein schlankes, sicheres Produktions-Image.

4.1 Images verwalten

```
docker pull python:3.14-slim      # Image laden, ohne einen Container zu starten
docker image ls                  # lokale Images (Kurzform: docker images)
docker image history python:3.14-slim # Schichten mit Größe und erzeugender Anweisung
docker image inspect python:3.14-slim # Details: Architektur, Umgebung, Digest
docker tag notizen:dev notizen:1.4.0 # zusätzlichen Namen vergeben (keine Kopie)
docker image rm notizen:dev       # Namen entfernen, Image löschen, wenn kein Name mehr bleibt
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
notizen	1.4.0	4c1f0a9d2e11	2 minutes ago	212MB
notizen	dev	4c1f0a9d2e11	2 minutes ago	212MB
python	3.14-slim	a7d3c1e2b9f0	5 days ago	148MB

Zwei Namen mit derselben Image-ID sind **ein** Image mit zwei Etiketten, genau wie zwei Branches, die auf denselben Commit zeigen [47].

4.2 Das erste Dockerfile

Das Beispielprojekt definiert seine Abhängigkeiten in `pyproject.toml` und hält die exakten Versionen in `uv.lock` fest [48]:

pyproject.toml

```
[project]
name = "notizen"
version = "1.4.0"
requires-python = ">=3.14"
dependencies = [
    "fastapi[standard]>=0.115",
    "psycopg[binary]>=3.2",
]
```

Dockerfile

```
FROM python:3.14-slim

# uv aus dem offiziellen Image übernehmen (in Kapitel 8 auf eine feste Version pinnen)
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

WORKDIR /app

# 1. Nur die Abhängigkeitsbeschreibung kopieren und installieren
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-install-project --no-dev

# 2. Danach den Code kopieren
```

```
COPY app/ ./app/

ENV PATH="/app/.venv/bin:$PATH"
EXPOSE 8000
CMD ["fastapi", "run", "app/main.py", "--port", "8000"]
```

Bauen und starten:

```
docker build -t notizen:dev .           # der Punkt ist der Build-Kontext (siehe unten)
docker run --rm -p 8000:8000 notizen:dev
curl http://localhost:8000/health      # in einem zweiten Terminal
```

4.3 Die Anweisungen

Anweisung	Bedeutung
FROM image	Basis-Image. Jedes Dockerfile beginnt damit (bei Multi-Stage-Builds mehrfach, Kapitel 8).
WORKDIR /app	Arbeitsverzeichnis für alle folgenden Anweisungen und für den Container, wird bei Bedarf angelegt.
COPY quelle ziel	Dateien aus dem Build-Kontext ins Image kopieren. <code>--from=</code> kopiert aus einem anderen Image oder einer Build-Stufe.
ADD	Wie <code>COPY</code> , kann zusätzlich URLs laden und Archive entpacken. Nur verwenden, wenn genau das gebraucht wird [19], [49].
RUN befehl	Befehl beim Bauen ausführen, das Ergebnis wird zur neuen Schicht.
ENV NAME=wert	Umgebungsvariable, gilt beim Bauen und im laufenden Container.
ARG NAME=wert	Variable nur beim Bauen, setzbar mit <code>docker build --build-arg NAME=...</code> .
EXPOSE 8000	Dokumentiert den Port. Veröffentlicht ihn nicht , das macht erst <code>-p</code> [49].
USER app	Benutzer für folgende Anweisungen und den Container (Kapitel 8).
CMD [...]	Standardbefehl beim Start, beim <code>docker run</code> überschreibbar.
ENTRYPOINT [...]	Fester Startbefehl, <code>CMD</code> liefert dann nur dessen Standardargumente.
HEALTHCHECK	Befehl, mit dem Docker regelmäßig prüft, ob der Container gesund ist (Kapitel 7).
LABEL	Metadaten wie Version oder Quell-Repository.

4.4 Build-Kontext und `.dockerignore`

Der Punkt am Ende von `docker build -t notizen:dev .` ist der **Build-Kontext**: das Verzeichnis, dessen Inhalt an den Build übergeben wird. Nur Dateien darin kann `COPY` erreichen [50]. Ohne Filter landen auch `.git`, virtuelle Umgebungen, Caches und im schlimmsten Fall `.env`-Dateien mit Passwörtern im Kontext und über ein unbedachtes `COPY . .` im Image. Eine `.dockerignore` schließt sie aus [19], [50]:

.dockerignore

```
.git
.venv
__pycache__/
*.pyc
.pytest_cache/
.env
.env.*
compose*.yaml
Dockerfile
```

4.5 Der Schicht-Cache

Docker merkt sich jede gebaute Schicht. Ändert sich an einer Anweisung und ihren Eingaben nichts, wird die Schicht aus dem Cache genommen. Sobald sich aber eine Schicht ändert, werden **alle folgenden** neu gebaut [51]. Deshalb kommt es auf die Reihenfolge an:

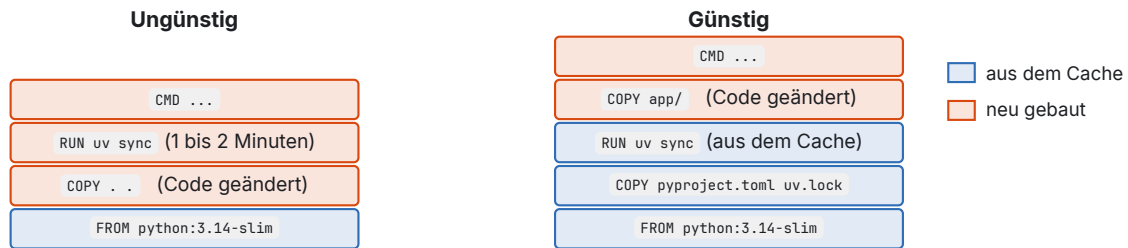


Abb. 7: Nach einer Codeänderung: Links werden alle Abhängigkeiten neu installiert, rechts nur der Code kopiert.

Die Faustregel: **Was sich selten ändert, gehört nach oben; was sich oft ändert, nach unten.** Deshalb kopiert das Dockerfile oben zuerst nur `pyproject.toml` und `uv.lock`, installiert die Abhängigkeiten und kopiert erst danach den Code [3], [52]. Eine Codeänderung baut dann in Sekunden.

4.6 CMD, ENTRYPOINT und die Signale

Beide Anweisungen gibt es in zwei Schreibweisen, und der Unterschied ist wichtiger, als er aussieht:

Form	Beispiel	Folge
Exec-Form	<code>CMD ["fastapi", "run", "app/main.py"]</code>	Das Programm ist direkt PID 1 und bekommt Signale wie SIGTERM (als PID 1 muss es SIGTERM allerdings selbst behandeln) [13], [49]. Immer verwenden.
Shell-Form	<code>CMD fastapi run app/main.py</code>	Docker startet <code>/bin/sh -c "..."</code> . PID 1 ist die Shell, die SIGTERM nicht weiterreicht [49].

`docker stop` schickt dem Hauptprozess SIGTERM und wartet zehn Sekunden. Reagiert er nicht, folgt SIGKILL [40]. Bei der Shell-Form kommt das SIGTERM in der Regel nicht bei der Anwendung an [49]: Jeder Stopp dauert dann zehn Sekunden und endet mit einem harten Abbruch, offene Datenbankverbindungen werden nicht sauber geschlossen.

`CMD` und `ENTRYPOINT` arbeiten zusammen: Ist ein `ENTRYPOINT` gesetzt, wird `CMD` als Argumente an ihn angehängt [49]. Das Muster eignet sich für Images, die wie ein Programm benutzt werden:

```
ENTRYPOINT ["python", "-m", "app.cli"]
CMD ["--help"]
```

```
docker run --rm notizen-cli          # führt "python -m app.cli --help" aus
docker run --rm notizen-cli import x.csv # ersetzt nur CMD: "... app.cli import x.csv"
```

Für die meisten Dienste genügt ein `CMD` in Exec-Form. Mit `docker run --entrypoint sh ...` lässt sich ein `ENTRYPOINT` zur Fehlersuche übergehen [35] (Kapitel 10).

5 Daten: Volumes und Bind Mounts

Aus Kapitel 1 ist klar: Die Schreibschicht eines Containers verschwindet mit ihm [15]. Bei jedem Update wird ein Container aber gerade gelöscht und aus dem neuen Image neu erzeugt. Alles, was bleiben soll, muss deshalb **außerhalb** des Containers liegen und nur hineingereicht werden. Dafür gibt es drei Mechanismen.

5.1 Drei Arten, Daten einzubinden

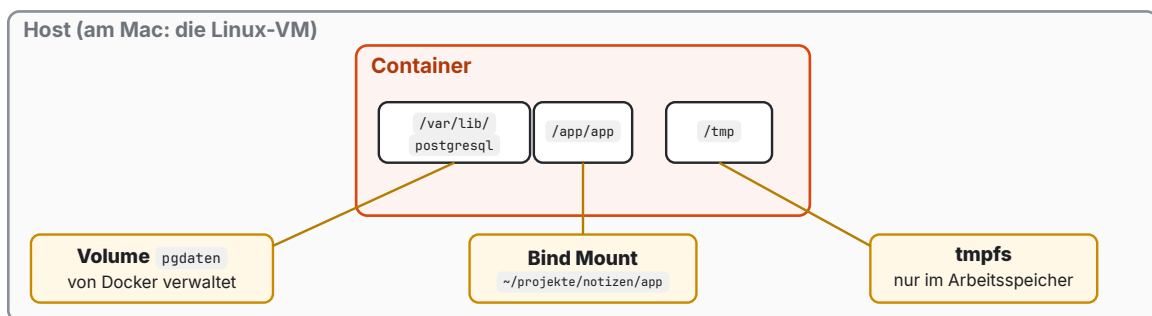


Abb. 8: Ein Container, drei Arten eingebundener Daten.

Art	Eigenschaften	Typische Verwendung
Volume	Von Docker verwaltet. Beim lokalen rootful Treiber liegt es typischerweise unter <code>/var/lib/docker/volumes</code> [36]; Rootless-Betrieb, VM und andere Volume-Treiber können andere Pfade nutzen.	Datenbanken, Uploads: alles Persistente in Produktion
Bind Mount	Ein beliebiges Verzeichnis des Hosts wird direkt eingeblendet. Änderungen sind sofort auf beiden Seiten sichtbar [53].	Quellcode während der Entwicklung, Konfigurationsdateien
tmpfs	Nur im Arbeitsspeicher, verschwindet beim Stopp [54].	temporäre Dateien, die nicht auf die Platte sollen

5.2 Volumes

```

docker volume create pgdaten      # anlegen (passiert bei -v auch automatisch)
docker volume ls                  # alle Volumes
docker volume inspect pgdaten     # Details, u.a. Pfad auf dem Host
docker volume rm pgdaten          # löschen (nur wenn kein Container es nutzt)

docker run -d --name db -v pgdaten:/var/lib/postgresql -e POSTGRES_PASSWORD=geheim postgres:18
# ausführlichere, gleichwertige Schreibweise:
docker run -d --name db --mount type=volume,src=pgdaten,dst=/var/lib/postgresql ... postgres:18

```

Die Syntax `-v a:b` ist mehrdeutig: Beginnt `a` mit `/` oder `.`, ist es ein Bind Mount, sonst ein Volume-Name [36]. `--mount` ist länger, aber eindeutig und meldet einen Fehler, wenn ein Host-Pfad nicht existiert, statt stillschweigend ein leeres Verzeichnis anzulegen [53].

Anonyme Volumes

Viele Images deklarieren mit `VOLUME` im Dockerfile einen Pfad für Daten. Bindet man dort nichts ein, legt Docker automatisch ein **anonymes** Volume mit einem Zufallsnamen an [36], [49]. Die Daten sind dann

zwar da, aber nach dem Neuerzeugen des Containers hängt ein **neues** anonymes Volume an diesem Pfad, und die alten Daten liegen verwaist in einem Volume namens `3f9a0c...` [36]. Daher gilt: Für persistente Daten immer ein **benanntes** Volume angeben.

ACHTUNG

PostgreSQL 18 hat den Datenpfad geändert. Bis Version 17 bindet man das Volume an `/var/lib/postgresql/data` [43]. Ab dem offiziellen Image für Version 18 liegt das Datenverzeichnis unter `/var/lib/postgresql/18/docker`, das deklarierte Volume ist `/var/lib/postgresql` [43], [55]. Wer bei `postgres:18` wie gewohnt `/var/lib/postgresql/data` einbindet, bekommt eine Fehlermeldung; ohne ein Volume auf `/var/lib/postgresql` landen die Daten in einem anonymen Volume, das beim Neuerzeugen verwaist [43], [56]. Also: bei 18+ an `/var/lib/postgresql` einbinden. Ein Wechsel der Hauptversion (17 -> 18) passiert zudem nie automatisch durch einen neuen Tag: Dafür ist ein Dump und Restore (siehe unten) oder `pg_upgrade` nötig [57].

5.3 Bind Mounts in der Entwicklung

Beim Entwickeln will man den Code nicht nach jeder Änderung neu bauen. Ein Bind Mount blendet das Projektverzeichnis direkt in den Container ein [53], zusammen mit einem Server, der bei Änderungen neu lädt:

```
docker run --rm -p 8000:8000 \
  -v "$PWD/app:/app/app" \
  notizen:dev fastapi dev app/main.py --host 0.0.0.0 --port 8000
```

Mit `:ro` am Ende (`-v "$PWD/config.toml:/app/config.toml:ro"`) ist der Mount im Container schreibgeschützt [53], was sich für Konfigurationsdateien empfiehlt. Komfortabler löst Compose das Thema mit `develop: watch` (Kapitel 7).

AUS DER PRAXIS

Auf Linux-Servern haben Mounts eine typische Falle: Dateien, die ein Container als `root` anlegt, gehören auf dem Host ebenfalls `root`, und ein Prozess, der im Container als Benutzer mit UID 1000 läuft, darf ein Verzeichnis von UID 1001 nicht beschreiben. Benutzer-IDs sind im Container und auf dem Host dieselben Zahlen, die Namen können sich unterscheiden. Am Mac übersetzt `virtiofs` einen Teil dieser Unterschiede, deshalb fällt das Problem oft erst auf dem Server auf. Auch ein neues benanntes Volume ist nicht automatisch für den Anwendungsbenutzer beschreibbar: Das Image muss es beim Start initialisieren, der Administrator muss Besitzer und Modus passend setzen, oder Host und Container verwenden abgestimmte UID/GID.

5.4 Sichern und Wiederherstellen

Für Datenbanken ist ein logischer Dump die sichere Wahl, weil er einen konsistenten Stand liefert, während die Datenbank weiterläuft [58], [59]:

```
# Sichern (Custom-Format, komprimiert)
docker exec db pg_dump -U notizen -Fc notizen > notizen-$(date +%F).dump

# Wiederherstellen in eine laufende, leere oder zu überschreibende Datenbank
docker exec -i db pg_restore -U notizen -d notizen --clean --if-exists < notizen-2026-09-24.dump
```

Bellebige Volumes lassen sich mit einem kurzlebigen Hilfscontainer als Archiv sichern [36]. Der Trick: Beide, das Volume und ein Verzeichnis des Hosts, werden in einen Wegwerf-Container eingebunden, der nur `tar` ausführt:

```
# Sichern: Volume "uploads" nach ./uploads.tgz
docker run --rm -v uploads:/daten:ro -v "$PWD":/backup alpine \
  tar czf /backup/uploads.tgz -C /daten .

# Wiederherstellen
```

```
docker run --rm -v uploads:/daten -v "$PWD":/backup alpine \  
tar xzf /backup/uploads.tgz -C /daten
```

ACHTUNG

Die Dateien einer **laufenden** Datenbank per `tar` zu kopieren, ergibt ein inkonsistentes Backup, das sich eventuell nicht wiederherstellen lässt [60]. Entweder die Datenbank vorher stoppen oder, besser, `pg_dump` verwenden. Und ein Backup, dessen Wiederherstellung nie getestet wurde, ist nur eine Hoffnung.

`pg_dump` sichert eine einzelne Datenbank. Globale Objekte wie Rollen und Tablespaces sind darin nicht enthalten [58]; falls sie nicht anderweitig als Konfiguration vorliegen, werden sie zusätzlich mit `pg_dumpall --globals-only` gesichert [61]. Produktionsbackups gehören verschlüsselt auf einen zweiten Rechner und werden regelmäßig probeweise wiederhergestellt.

6 Netzwerke

Jeder Container hat ein eigenes Netzwerk-Interface mit eigener IP-Adresse [62]. Wie Container einander finden und wie sie von außen erreichbar werden, bestimmen Docker-Netzwerke und veröffentlichte Ports. Die beiden häufigsten Fragen, „Warum erreicht meine App die Datenbank nicht?“ und „Warum ist der Port von außen nicht erreichbar?“, haben hier ihre Antwort.

6.1 Eigene Netzwerke und Namensauflösung

Ohne weitere Angabe landen Container im Standardnetz `bridge`. Dort erreichen sie sich nur über IP-Adressen, die sich bei jedem Neustart ändern können. In einem **selbst angelegten** Netz dagegen löst Docker die Container-Namen automatisch als Hostnamen auf [62]:

```
docker network create notizen-netz
docker run -d --name db --network notizen-netz -v pgdaten:/var/lib/postgresql \
-e POSTGRES_PASSWORD=geheim postgres:18
docker run -d --name api --network notizen-netz -p 8000:8000 \
-e DATABASE_URL=postgresql://postgres:geheim@db:5432/postgres notizen:dev
```

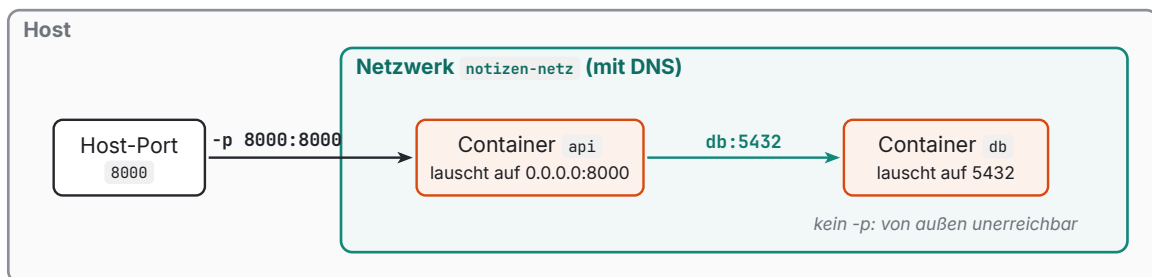


Abb. 9: Die App erreicht die Datenbank über ihren Namen `db`. Nur die App hat einen veröffentlichten Port.

Das ist das Grundmuster fast jeder Anwendung: Alle zusammengehörigen Container teilen sich ein eigenes Netz und sprechen sich mit Namen an. Nur der Container, der von außen erreichbar sein muss, veröffentlicht einen Port. Die Datenbank bleibt intern. Docker Compose legt ein solches Netz pro Projekt automatisch an (Kapitel 7) [63].

ACHTUNG

Innerhalb eines Containers bedeutet `localhost` **der Container selbst**, nicht der Host und nicht ein anderer Container. `DATABASE_URL=...@localhost:5432` funktioniert deshalb in Docker nicht, obwohl es ohne Docker klappte. Richtig ist der Name des Datenbank-Containers.

6.2 Ports veröffentlichen

`-p Host-Port:Container-Port` leitet Verbindungen zu einem Port des Hosts an den Container weiter [64]. Dabei zählt auch die Adresse:

Angabe	Erreichbar von
<code>-p 8000:8000</code>	allen Netzwerkschnittstellen des Hosts, also auch aus dem Internet, falls der Host dort hängt
<code>-p 127.0.0.1:8000:8000</code>	nur vom Host selbst. Richtig für Dienste hinter einem Reverse Proxy und für Datenbanken, die man lokal mit einem GUI-Werkzeug ansehen will.
<code>-p 8080:80</code>	Host-Port 8080 wird auf Port 80 im Container geleitet. Die Nummern müssen nicht gleich sein.

Eine zweite, häufige Ursache für „nicht erreichbar“ liegt in der Anwendung selbst: Sie muss im Container auf `0.0.0.0` lauschen, nicht auf `127.0.0.1`. Ein Server, der nur auf `127.0.0.1` lauscht, nimmt ausschließlich Verbindungen aus dem Container selbst an, und die Weiterleitung von Docker kommt nie an. `fastapi run` lauscht bereits auf `0.0.0.0`, `uvicorn` dagegen standardmäßig auf `127.0.0.1`, dort ist `--host 0.0.0.0` nötig [65], [66].

ACHTUNG

Veröffentlichte Ports umgehen die Firewall `ufw`. Auf Linux-Servern trägt Docker eigene Regeln in die Paketfilter des Kernels ein, die vor den Regeln von `ufw` greifen [67]. Ein `-p 5432:5432` macht die Datenbank deshalb aus dem Internet erreichbar, auch wenn `ufw` Port 5432 angeblich sperrt. Abhilfe: Dienste nur an `127.0.0.1` binden und über einen Reverse Proxy veröffentlichen (Kapitel 13), zusätzlich die Firewall des Hosting-Anbieters nutzen.

6.3 Vom Container zum Host

Manchmal muss ein Container einen Dienst erreichen, der direkt auf dem Host läuft, etwa eine lokal installierte Datenbank. Für `--add-host` bietet Docker den besonderen Wert `host-gateway`, der auf die Adresse des Hosts zeigt [35]. Der frei gewählte Hostname links davon lautet im Beispiel `host.docker.internal`:

```
docker run --rm --add-host=host.docker.internal:host-gateway alpine \
  wget -q0- http://host.docker.internal:8080
```

Docker Desktop richtet `host.docker.internal` automatisch ein [68], auf Linux-Servern braucht es das `--add-host` (in Compose: `extra_hosts`) [69]. Aktuelle Colima-Versionen stellen den Namen ebenfalls bereit [70]; bei älteren Profilen und besonderen Netzwerkmodi kann das Verhalten abweichen. Am portabelsten ist es, auch den Zieldienst als Container im selben Netz zu betreiben.

6.4 Netzwerktreiber

Treiber	Verwendung
<code>bridge</code>	Standard. Eigenes, privates Netz auf einem Host. Für fast alles richtig [71].
<code>host</code>	Container nutzt direkt das Netz des Hosts, ohne Isolation und ohne <code>-p</code> . Volle Wirkung nur auf Linux. Docker Desktop bietet es ab Version 4.34 als optionale Einstellung [72], bei Colima ist „Host“ die VM, nicht der Mac.
<code>none</code>	kein Netzwerk, etwa für reine Rechenjobs
<code>overlay</code>	Netz über mehrere Hosts hinweg, nur mit Swarm (Kapitel 15) [73]

6.5 Netzwerke untersuchen

```
docker network ls
docker network inspect notizen-netz          # welche Container mit welcher IP
docker run --rm -it --network notizen-netz nicolaka/netshoot # Werkzeugkasten im selben Netz
```

Das Image `nicolaka/netshoot` enthält alle üblichen Netzwerkwerkzeuge (`dig`, `curl`, `nc`, `ping`, `tcpdump`) [74]. In dasselbe Netz gestartet, lässt sich damit prüfen, ob ein Name aufgelöst wird (`dig db`) und ob ein Port antwortet (`nc -zv db 5432`), ohne die eigenen Images mit Werkzeugen zu belasten.

7 Docker Compose

Spätestens mit zwei Containern, einem Netz und einem Volume werden `docker run`-Befehle unhandlich. Docker Compose beschreibt die ganze Anwendung **deklarativ** in einer Datei: welche Dienste es gibt, aus welchen Images sie entstehen, wie sie verbunden sind und welche Daten sie behalten. Ein einziger Befehl bringt dann alles in genau diesen Zustand.

7.1 Die erste `compose.yaml`

`compose.yaml`

```
services:
  api:
    build: .                # Image aus dem Dockerfile im Projekt bauen
    image: notizen:dev
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      DATABASE_URL: postgresql://notizen:${DB_PASSWORD}@db:5432/notizen
    depends_on:
      db:
        condition: service_healthy      # erst starten, wenn die DB bereit ist
    restart: unless-stopped

  db:
    image: postgres:18
    environment:
      POSTGRES_USER: notizen
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: notizen
    volumes:
      - pgdaten:/var/lib/postgresql      # Pfad ab PostgreSQL 18 (Kapitel 5)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U notizen -d notizen"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

volumes:
  pgdaten:
```

`.env`

```
DB_PASSWORD=entwicklung
```

```
docker compose up -d      # alles bauen (falls nötig), anlegen und starten
docker compose ps
curl http://localhost:8000/health
```

Die Datei heißt `compose.yaml` (ältere Projekte nutzen `docker-compose.yaml`, beides wird erkannt) [75]. Eine Zeile `version: "3.8"` am Anfang ist veraltet und wird nur noch mit einer Warnung ignoriert [76].

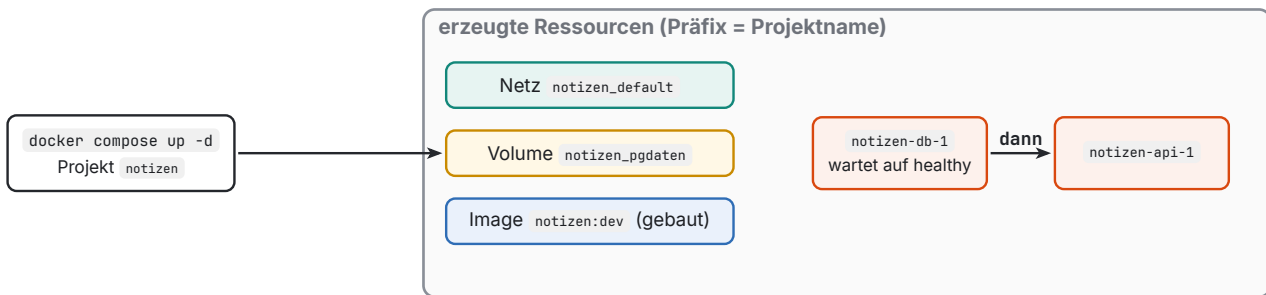


Abb. 10: Was `docker compose up` anlegt. Der Projektname ist standardmäßig der Verzeichnisname.

Compose leitet einen **Projektnamen** ab, standardmäßig den Namen des Verzeichnisses, und stellt ihn allen Ressourcen voran [77]. Das Netz `notizen_default` entsteht automatisch, alle Dienste der Datei hängen darin und erreichen sich über ihren Dienstenamen (`db`, `api`) [63]. Zwei Projekte mit gleichem Verzeichnisnamen würden sich dieselben Ressourcen teilen. Dann hilft `name: notizen` oben in der Datei oder `-p name` beim Aufruf.

7.2 Die Befehle

Befehl	Wirkung
<code>docker compose up -d</code>	Sollzustand herstellen: fehlende Images bauen, geänderte Container neu erzeugen, starten [78]
<code>docker compose up -d --build</code>	vor dem Start einen Build anstoßen; unveränderte Schichten dürfen aus dem Cache kommen
<code>docker compose up -d --wait</code>	starten und bis <code>running</code> beziehungsweise <code>healthy</code> warten
<code>docker compose ps</code>	Dienste mit Status und Gesundheit
<code>docker compose logs -f api</code>	Ausgaben eines Dienstes (ohne Name: aller Dienste, farbig getrennt)
<code>docker compose exec db psql -U notizen</code>	Befehl in einem laufenden Dienst ausführen
<code>docker compose run --rm api pytest</code>	Einmal-Container eines Dienstes mit anderem Befehl starten
<code>docker compose restart api</code>	Dienst neu starten (ohne Neuerzeugung, Konfigurationsänderungen greifen nicht)
<code>docker compose stop / start</code>	anhalten, ohne etwas zu löschen
<code>docker compose down</code>	Container und Netz entfernen. Volumes und Images bleiben.
<code>docker compose down -v</code>	zusätzlich die Volumes löschen , also die Datenbank! [46]
<code>docker compose pull</code>	neuere Versionen der Images laden (danach <code>up -d</code>)
<code>docker compose config</code>	die vollständig aufgelöste Konfiguration anzeigen [79]

MERKE

`docker compose up -d` ist **idempotent**: Es vergleicht Soll- und Istzustand und ändert nur, was nötig ist. Nach jeder Änderung an `compose.yaml`, `.env` oder am Image genügt ein erneutes `up -d`. Ein `restart` übernimmt dagegen keine Konfigurationsänderungen, weil der Container nicht neu erzeugt wird.

7.3 Variablen: `.env` und `env_file`

Hier gibt es eine Verwechslungsgefahr, die fast jeder einmal erlebt. Es gibt zwei völlig verschiedene Mechanismen:

	<code>.env</code> im Projektordner	<code>env_file</code> : in einem Dienst
Wirkt auf	die <code>compose.yaml</code> selbst: ersetzt <code>\${VARIABLE}</code> beim Einlesen [80]	den Container: setzt Umgebungsvariablen im laufenden Prozess [81]
Automatisch?	aus dem Projektverzeichnis; mit <code>--env-file</code> lässt sich die Quelle explizit wählen	nur wenn im Dienst angegeben
Sichtbar im Container?	nur, was über <code>environment</code> : weitergereicht wird	ja, alle Einträge

```
environment:
  LOG_LEVEL: ${LOG_LEVEL:-info}           # mit Standardwert
  SECRET_KEY: ${SECRET_KEY:?SECRET_KEY fehlt} # Abbruch mit Meldung, wenn nicht gesetzt
env_file:
  - app.env                               # alle Zeilen als Variablen in den Container
```

Shell-Variablen, explizite `--env-file`-Dateien und `.env` haben eine festgelegte Priorität [80]. Was Compose aus allen Quellen gemacht hat, zeigen `docker compose config` und `docker compose config --environment`. Das ist der erste Schritt, wenn ein Wert nicht ankommt. Geheimnisse gehören trotzdem nicht in `.env`, weil die aufgelöste Konfiguration und Prozessumgebung sie offenlegen können [81].

7.4 Startreihenfolge, Healthchecks und Migrationen

`depends_on` allein regelt nur die **Startreihenfolge**: Die Datenbank wird zuerst gestartet, ist aber in dem Moment noch lange nicht bereit, Verbindungen anzunehmen. Erst `condition: service_healthy` wartet beim Start, bis der Healthcheck des anderen Dienstes erfolgreich ist [69]. Das ist keine dauerhafte Selbstheilung: Wird ein laufender Container später `unhealthy`, startet ihn Docker deshalb nicht automatisch neu, denn der Healthcheck setzt nur den Status. Eine Restart Policy greift erst, wenn sein Hauptprozess endet [37], [49]. Deshalb braucht jede Datenbank im Compose-Projekt einen Healthcheck, und die Anwendung muss vorübergehende Verbindungsfehler selbst mit Retries behandeln.

Datenbank-Migrationen, etwa mit Alembic, laufen am saubersten als eigener Einmal-Dienst, der vor der API fertig sein muss:

```
migrate:
  image: notizen:dev
  command: ["alembic", "upgrade", "head"]
  environment:
    DATABASE_URL: postgresql://notizen:${DB_PASSWORD}@db:5432/notizen
  depends_on:
    db:
      condition: service_healthy
  restart: "no"

api:
  # ...
  depends_on:
    migrate:
      condition: service_completed_successfully
```

Schlägt die Migration fehl, startet die API gar nicht erst. Das ist genau richtig, denn eine neue Version gegen ein altes Datenbankschema laufen zu lassen, führt zu schwer verständlichen Fehlern.

Seit Compose 5.3 kann ein Dienst alternativ über `pre_start` einen Initialisierungsschritt unmittelbar vor seinem Hauptprozess ausführen [82], [83]. Für kleine lokale Setups ist das kompakt. Ein eigener Migrations-Dienst bleibt für Produktion oft klarer, weil sein einmaliger Status sichtbar ist, er von mehreren Diensten abhängig sein kann und sich unabhängig aufrufen lässt [83].

7.5 Entwickeln mit Compose Watch

Für die Entwicklung soll eine Codeänderung sofort wirken, ohne Neubau. `develop: watch` synchronisiert geänderte Dateien in den laufenden Container und baut nur neu, wenn sich die Abhängigkeiten ändern [84]:

```
api:
  build: .
  command: ["fastapi", "dev", "app/main.py", "--host", "0.0.0.0", "--port", "8000"]
  develop:
    watch:
      - action: sync                # Code: Dateien in den Container kopieren
        path: ./app
        target: /app/app
```

```
- action: rebuild      # Abhängigkeiten: Image neu bauen
  path: uv.lock
```

```
docker compose up --watch      # startet alles und überwacht die Pfade
```

`fastapi dev` lädt die Anwendung bei jeder synchronisierten Änderung neu. Gegenüber einem Bind Mount hat das den Vorteil, dass keine Dateien vom Container zurück auf den Mac geschrieben werden und das Verhalten auf allen Rechnern gleich ist [84].

7.6 Profile: optionale Dienste

Werkzeuge wie eine Datenbank-Oberfläche sollen nur bei Bedarf laufen. Dienste mit einem Profil startet Compose nur, wenn das Profil ausdrücklich aktiviert wird [85]:

```
adminer:
  image: adminer
  ports: ["127.0.0.1:8081:8080"]
  profiles: [werkzeuge]
```

```
docker compose --profile werkzeuge up -d
```

7.7 Entwicklung und Produktion trennen

Die `compose.yaml` im Repository ist auf die Entwicklung ausgelegt: Sie baut das Image selbst, veröffentlicht die API direkt und nutzt ein einfaches Passwort. Auf dem Server gelten andere Regeln: Das Image kommt fertig aus der Registry, davor steht ein Reverse Proxy, Passwörter liegen in Dateien. Dafür gibt es zwei Wege:

- **Eine eigene Datei für die Produktion**, im Repository als `compose.prod.yaml` gepflegt und auf dem Server als `compose.yaml` abgelegt. Übersichtlich, weil man auf dem Server genau sieht, was läuft. Diesen Weg nutzt Kapitel 13.
- **Überlagerung mehrerer Dateien**: `docker compose -f compose.yaml -f compose.prod.yaml up -d` führt beide zusammen, spätere Dateien überschreiben frühere. Eine Datei `compose.override.yaml` wird sogar automatisch dazugenommen [86]. Das spart Wiederholungen, macht aber schwerer nachvollziehbar, was am Ende gilt. Dann hilft wieder `docker compose config`.

8 Gute Images bauen

Das Image aus Kapitel 4 funktioniert, hat aber drei Schwächen: Es enthält Werkzeuge, die zur Laufzeit niemand braucht (uv, Build-Caches), der Prozess läuft als `root`, und Versionen sind nicht festgelegt. Dieses Kapitel baut daraus das Produktions-Image des Beispielprojekts.

8.1 Das Basis-Image wählen

Variante	Größe	Einordnung
<code>python:3.14</code>	ca. 1 GB	Volles Debian mit Compilern. Nur als Build-Stufe sinnvoll, wenn Pakete kompiliert werden müssen.
<code>python:3.14-slim</code>	ca. 150 MB	Schlankes Debian mit Python [87]. Guter Standard.
<code>python:3.14-alpine</code>	ca. 50 MB	Sehr klein, nutzt aber die C-Bibliothek musl statt glibc [87]. Binärpakete (Wheels) müssen dafür als <code>musllinux</code> gebaut sein [88]; fehlt eines, wird aus dem Quellcode kompiliert, und Software mit glibc-Anahmen kann sich anders verhalten. Für Python meist nicht die Ersparnis wert.
<i>distroless</i> / gehärtete Images	klein	Nur Laufzeit, keine Shell, kein Paketmanager [42]. Minimale Angriffsfläche, aber schwerer zu debuggen.

Die Größenangaben sind Größenordnungen für entpackte Images (wie `docker image ls` sie zeigt), Docker Hub nennt kleinere, komprimierte Werte. Wichtiger als die letzten Megabytes ist, dass das Image nichts enthält, was nicht gebraucht wird: jedes zusätzliche Programm ist potenziell eine Sicherheitslücke [19].

8.2 Multi-Stage-Builds

Ein Dockerfile kann mehrere `FROM`-Anweisungen enthalten. Jede beginnt eine neue **Stufe**. Nur die letzte Stufe wird zum Image, aus früheren Stufen werden gezielt Ergebnisse herüberkopiert [89]. So bleiben Build-Werkzeuge und Caches draußen.

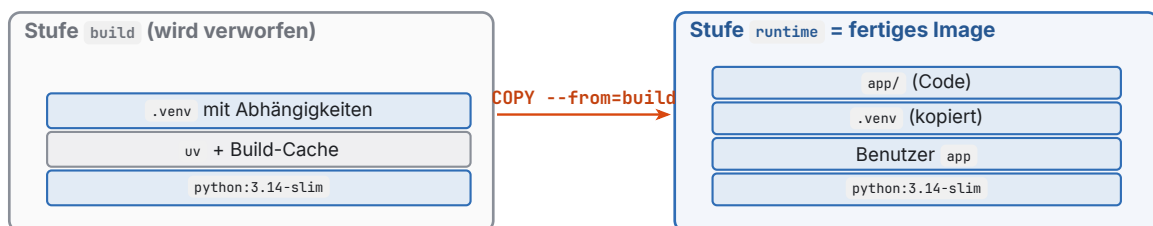


Abb. 11: Nur die virtuelle Umgebung wandert in das fertige Image, uv und der Cache bleiben in der Build-Stufe.

8.3 Das Produktions-Dockerfile

Dockerfile

```
# syntax=docker/dockerfile:1
ARG PYTHON_VERSION=3.14
ARG UV_VERSION=0.12.17                                # feste Version (Stand 18.09.2026)

FROM ghcr.io/astral-sh/uv:${UV_VERSION} AS uv

# ----- Stufe 1: Abhängigkeiten installieren -----
FROM python:${PYTHON_VERSION}-slim AS build
COPY --from=uv /uv /bin/uv
```

```

ENV UV_COMPILE_BYTECODE=1 \
    UV_LINK_MODE=copy \
    UV_PYTHON_DOWNLOADS=never
WORKDIR /app
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    uv sync --locked --no-install-project --no-dev

# ----- Stufe 2: schlankes Laufzeit-Image -----
FROM python:${PYTHON_VERSION}-slim AS runtime
RUN groupadd --system --gid 10001 app \
    && useradd --system --uid 10001 --gid app --no-create-home app
WORKDIR /app
COPY --from=build --chown=app:app /app/.venv /app/.venv
COPY --chown=app:app app/ ./app/
ENV PATH="/app/.venv/bin:$PATH" \
    PYTHONUNBUFFERED=1
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
    CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health', timeout=2)"]
CMD ["fastapi", "run", "app/main.py", "--port", "8000"]

```

Was die einzelnen Teile bewirken:

- `# syntax=docker/dockerfile:1` verwendet die aktuelle Dockerfile-Syntax. Die `--mount`-Optionen setzen Dockerfile-Syntax 1.2 oder neuer voraus [49].
- **ARG vor dem ersten FROM** macht Versionen an einer Stelle änderbar, auch von außen [49]:
`docker build --build-arg PYTHON_VERSION=3.14 .`
- `--mount=type=cache` hält den Download-Cache von uv zwischen Builds vor, ohne dass er im Image landet [3], [49]. Neue Abhängigkeiten werden nicht jedes Mal komplett neu geladen.
- `--mount=type=bind` stellt `pyproject.toml` und `uv.lock` nur für diesen einen Schritt bereit, ohne eine eigene Schicht zu erzeugen [3].
- `UV_COMPILE_BYTECODE=1` erzeugt die `.pyc`-Dateien schon beim Bauen, das beschleunigt den Start [3], [90].
- **Eigener Benutzer mit fester UID** (Kapitel 12): Der Prozess läuft nicht als `root` [19]. Die feste Nummer erleichtert Rechte auf Volumes und Bind Mounts.
- `PYTHONUNBUFFERED=1` sorgt dafür, dass Ausgaben sofort in `docker logs` erscheinen und nicht in einem Puffer hängen.
- `HEALTHCHECK` lässt Docker selbst prüfen, ob die Anwendung antwortet [49]. Das slim-Image hat kein `curl` [87], deshalb erledigt Python den Aufruf.

Geheimnisse beim Bauen

Passwörter, Tokens und SSH-Schlüssel dürfen weder über `ARG` oder `ENV` noch mit `COPY` in den Build gelangen: Sie können in Layern, Metadaten oder Cache-Exporten erhalten bleiben [91]. BuildKit stellt sie nur für einen einzelnen `RUN`-Schritt bereit [49]:

```

RUN --mount=type=secret,id=pypi_token \
    TOKEN="$(cat /run/secrets/pypi_token)" uv sync --locked
# für private Git-Abhängigkeiten analog: RUN --mount=type=ssh ...

```

```

docker build --secret id=pypi_token,src="$HOME/.config/notizen/pypi-token" .

```

Das Geheimnis wird dabei nicht Teil des Images [91]. Der verwendete Befehl darf es allerdings auch nicht selbst in Dateien oder Logs schreiben.

8.4 Versionen festlegen

Reproduzierbare Builds brauchen feste Versionen auf jeder Ebene:

Ebene	Festlegung
Basis-Image	beweglicher Versions- und Varianten-Tag (<code>python:3.14-slim-bookworm</code>) oder unveränderlicher Digest (<code>python:3.14-slim-bookworm@sha256:...</code>) [19]
Werkzeuge	<code>ARG UV_VERSION=...</code> statt <code>latest</code>
Python-Pakete	<code>uv.lock</code> mit <code>uv sync --locked</code> : Der Build bricht ab, wenn Lockfile und <code>pyproject.toml</code> nicht zusammenpassen [48].
Systempakete	nur wenn nötig, dann mit <code>apt-get install --no-install-recommends</code> und anschließendem Aufräumen von <code>/var/lib/apt/lists</code> [19]

Ein Tag bleibt beweglich und liefert Aktualität, aber keine bitgenaue Reproduzierbarkeit. Ein Digest ist unveränderlich, hat jedoch einen Preis: Sicherheitsupdates des Basis-Images kommen nicht mehr automatisch [19]. Wer pinnt, braucht einen Prozess zum Aktualisieren, etwa Renovate mit Pull Requests für neue Digests [92]. Für kleine Projekte ist ein expliziter Python- und Debian-Tag plus regelmäßiges Neubauen pragmatisch; für kontrollierte Releases wird der getestete Digest dokumentiert oder direkt gepinnt.

TIPP

Als weitere Option gibt es *Docker Hardened Images*: minimale, standardmäßig als Nicht-Root ausgelegte Images mit SBOM, Provenance und Signaturen [93]. Sie reduzieren die Angriffsfläche, ersetzen aber keine Kompatibilitäts- und Funktionstests. Für das durchgängige Beispiel bleibt das offizielle Python-Image verständlicher.

8.5 Metadaten

```
LABEL org.opencontainers.image.source="https://gitea.example.com/team/notizen" \
      org.opencontainers.image.version="1.4.0" \
      org.opencontainers.image.revision="3f2a9c1"
```

Die standardisierten `org.opencontainers` -Labels verknüpfen ein Image mit seinem Quellcode [94]. Version und Commit setzt man in der CI per `--label` oder `--build-arg`, damit sie nicht von Hand gepflegt werden müssen (Kapitel 14).

9 ARM und x86: Builds für mehrere Plattformen

Ein Mac mit Apple Silicon ist ein ARM-Rechner (`arm64`), die meisten Server und CI-Runner sind x86-Rechner (`amd64`). Ein Image enthält Maschinencode für genau eine dieser Architekturen. Wer am Mac baut und das Ergebnis auf den Server schiebt, bekommt dort eine knappe Fehlermeldung:

```
exec /app/.venv/bin/fastapi: exec format error
```

Das heißt: Das Programm im Image ist für eine andere Prozessorarchitektur übersetzt. Bei Python betrifft das vor allem das Basis-Image und Pakete mit kompiliertem Code (etwa `psycopp[binary]`).

9.1 Plattform gezielt wählen

```
uname -m                                # am Mac: arm64
docker image inspect notizen:dev --format '{{.Os}}/{{.Architecture}}' # linux/arm64

docker build --platform linux/amd64 -t notizen:amd64 .      # für x86 bauen (emuliert)
docker run --rm --platform linux/amd64 notizen:amd64 uname -m # x86_64
```

Am Mac übernimmt Rosetta (Kapitel 2) oder QEMU die Emulation [28]. Das funktioniert zuverlässig, ist aber spürbar langsamer als ein nativer Build, bei großen Kompilierungen um ein Vielfaches [95].

9.2 Ein Name, mehrere Plattformen

Offizielle Images wie `python:3.14-slim` gibt es für viele Architekturen unter **einem** Namen [87]. Dahinter steckt ein *Image-Index* (auch *Manifest List*): Der Tag zeigt auf eine Liste [96], [97], und Docker lädt automatisch die Variante, die zum eigenen Rechner passt.

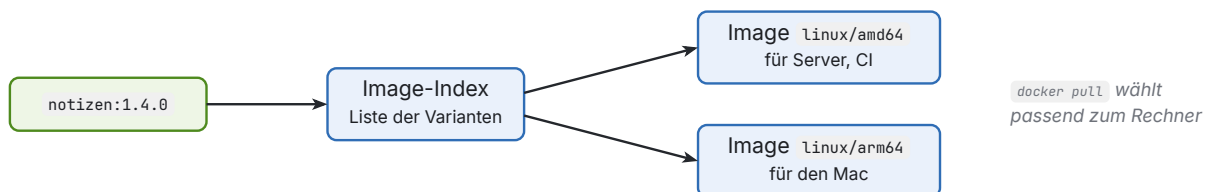


Abb. 12: Ein Tag kann auf einen Index zeigen, der je eine Variante pro Architektur enthält.

Eigene Multi-Plattform-Images baut `docker buildx` [95]. Bei frischen Installationen ab Docker Engine 29 ist der containerd Image Store Standard und kann einen mehrplattformigen Index auch lokal verwalten [4], [98]. Aktualisierte Installationen können weiterhin den klassischen Image-Speicher nutzen; dieser kann solche Indizes nicht lokal laden [98]. Ein `docker-container`-Builder mit direktem Push funktioniert in beiden Fällen und ist daher für CI gut geeignet [99]:

```
docker buildx create --name multi --driver docker-container --use # einmalig
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t gitea.example.com/team/notizen:1.4.0 \
  --push .
docker buildx imagetools inspect gitea.example.com/team/notizen:1.4.0 # Varianten prüfen
```

9.3 Empfehlung für die Praxis

- **Am Mac** nativ für `arm64` bauen und testen. Das ist schnell und deckt fast alle Fehler ab.

- **Für den Server** in der CI auf einem `amd64`-Runner bauen (Kapitel 14). Der baut nativ, schnell und ohne Emulation [95]. Nach dem Build wird genau dieses Image als Container getestet und anschließend unter seinem Digest veröffentlicht.
- **Multi-Plattform-Images** nur dann, wenn sie wirklich auf beiden Architekturen laufen müssen, etwa wenn Kollegen mit ARM-Macs das fertige Produktions-Image lokal starten sollen oder ein ARM-Server im Spiel ist.

TIPP

Wer eine Compose-Datei hat, die am Mac ein bestimmtes `amd64`-Image nutzen muss (etwa weil es kein ARM-Image gibt), kann die Plattform pro Dienst festlegen: `platform: linux/amd64`. Compose lädt dann diese Variante [69] und Colima führt sie über Rosetta aus [27].

10 Fehlersuche

Die meisten Docker-Probleme lassen sich mit einem immer gleichen Vorgehen eingrenzen. Wer die Schritte in dieser Reihenfolge abarbeitet, findet die Ursache fast immer, bevor er im Internet suchen muss.

10.1 Das Vorgehen



Abb. 13: Von außen nach innen: erst schauen, was Docker meldet, dann in den Container hinein.

- Status:** Läuft der Container, ist er neu gestartet worden (`Restarting`), beendet (`Exited (1)`) oder ungesund (`unhealthy`)? `docker compose ps -a` zeigt auch beendete Container.
- Logs:** Die letzte Fehlermeldung der Anwendung steht fast immer in `docker compose logs --tail 50 api`. Das löst die Mehrzahl aller Fälle.
- Konfiguration:** `docker compose config` zeigt, was nach dem Einsetzen aller Variablen tatsächlich gilt. Fehlende Werte, falsche Pfade und Tippfehler in Variablennamen fallen hier auf.
- Inspizieren:** `docker inspect` liefert den Zustand im Detail, etwa Exit-Code, Grund des Abbruchs, Healthcheck-Ergebnisse, Mounts und Netzwerke [41].
- Hineinsehen:** Mit einer Shell im Container prüfen, ob Dateien, Rechte, Umgebungsvariablen und Netzwerkverbindungen so sind wie erwartet.

```

docker inspect api --format '{{.State.Status}} {{.State.ExitCode}} OOM={{.State.OOMKilled}}'
docker inspect api --format '{{json .State.Health}}' | python3 -m json.tool
docker compose exec api env | sort # Umgebung im laufenden Container
docker compose run --rm --entrypoint sh api # Shell trotz Absturz der App
docker events --since 10m # was hat der Daemon zuletzt getan?
  
```

Der vorletzte Befehl ist der wichtigste Trick bei Containern, die sofort abstürzen: `exec` geht dann nicht, weil der Container nicht läuft. `run --entrypoint sh` startet stattdessen einen neuen Container aus demselben Image mit einer Shell statt der Anwendung [100]. Darin lässt sich der Startbefehl von Hand ausführen und die Fehlermeldung in Ruhe lesen.

10.2 Exit-Codes lesen

Code	Bedeutung
0	Der Prozess hat sich regulär beendet. Bei einem Dienst heißt das oft: Er hatte nichts zu tun (falscher Befehl, Programm läuft nicht im Vordergrund).
1	Allgemeiner Fehler der Anwendung, Details in den Logs.
125	Docker selbst konnte den Container nicht starten [101] (ungültige Option, Port belegt).
126	Startbefehl gefunden, aber nicht ausführbar (fehlende Rechte) [101], [102].
127	Startbefehl nicht gefunden (Tippfehler, falscher <code>PATH</code> , Programm nicht im Image) [101].
137	Mit SIGKILL beendet [103]: Speicherlimit überschritten (<code>OOMKilled=true</code>) [104] oder <code>docker stop</code> lief in den Timeout [40].
139	Speicherzugriffsfehler (<code>SIGSEGV</code>) [103]: typischerweise Fehler in nativem Code oder inkompatible Bibliothek. Eine falsche CPU-Architektur meldet meist <code>exec format error</code> .
143	Mit SIGTERM beendet, der Normalfall nach <code>docker stop</code> [40].

10.3 Häufige Probleme

Symptom	Ursache und Lösung
<i>Cannot connect to the Docker daemon</i>	Colima läuft nicht (<code>colima start</code>) oder der falsche Kontext ist aktiv (<code>docker context ls</code>) [32].
<i>port is already allocated</i>	Ein anderer Container oder Prozess nutzt den Host-Port. <code>docker ps</code> bzw. <code>lsof -i :8000</code> am Mac.
App erreicht die Datenbank nicht	<code>localhost</code> statt Dienstname verwendet, Container in verschiedenen Netzen, oder die DB war beim Start noch nicht bereit (Healthcheck plus <code>service_healthy</code>).
Port von außen nicht erreichbar	Anwendung lauscht im Container auf <code>127.0.0.1</code> statt <code>0.0.0.0</code> , oder <code>-p</code> fehlt bzw. ist an <code>127.0.0.1</code> gebunden.
Daten nach Update weg	Kein oder ein anonymes Volume, falscher Pfad (PostgreSQL 18: <code>/var/lib/postgresql</code>) [43], oder <code>down -v</code> verwendet.
Code-Änderung wirkt nicht	Image nicht neu gebaut (<code>up -d --build</code>) oder alter Container läuft noch, weil <code>restart</code> statt <code>up -d</code> verwendet wurde.
<i>exec format error</i>	Image für die falsche Architektur gebaut (Kapitel 9).
<i>permission denied</i> auf Dateien	Prozess läuft als Benutzer <code>app</code> , Dateien gehören <code>root</code> (Bind Mount oder <code>COPY</code> ohne <code>--chown</code>).
Container startet immer wieder neu	Absturz beim Start plus <code>restart: unless-stopped</code> . Logs lesen, zur Diagnose vorübergehend <code>restart: "no"</code> .
<i>no space left on device</i>	Platte der VM bzw. des Servers voll: <code>docker system df</code> , dann aufräumen (Kapitel 3) [33]. Bei Colima eventuell die Disk vergrößern.
Build ignoriert Änderungen	Schicht-Cache greift unerwartet, etwa bei <code>RUN git clone</code> oder heruntergeladenen Dateien. <code>docker build --no-cache</code> erzwingt einen vollständigen Neubau [105].

TIPP

Viele Images haben keine Werkzeuge wie `ps` , `curl` oder `ping` . Statt sie ins Image einzubauen, startet man einen Werkzeug-Container im selben Netz (`nicolaka/netshoot` , Kapitel 6) [74] oder für Prozess- und Netzsicht mit `docker run --rm -it --pid container:api --network container:api nicolaka/netshoot` [35]. Das teilt nicht das Dateisystem der Anwendung. Dateien sieht man nur über explizite Mounts, `--volumes-from` oder - mit ausreichenden Rechten - über `/proc/<PID>/root` [106].

11 Registries: Images speichern und verteilen

Damit ein Image vom Build-Rechner auf den Server kommt, braucht es eine Registry als Zwischenlager. Docker Hub ist die Standardquelle für öffentliche Images. Für eigene Images bietet sich die Container-Registry von Gitea an: Sie liegt neben dem Code, nutzt dieselben Benutzer und Rechte und kostet nichts extra.

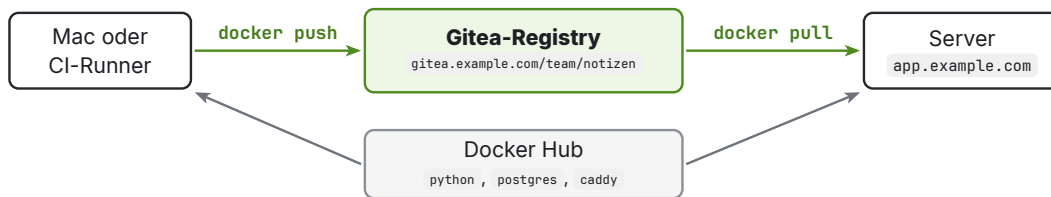


Abb. 14: Eigene Images wandern über die Gitea-Registry, Basis-Images kommen von Docker Hub.

11.1 Die Gitea-Registry

Images in Gitea heißen `gitea.example.com/<besitzer>/<image>:<tag>`, wobei der Besitzer ein Benutzer oder eine Organisation ist [107]. Zum Anmelden braucht es einen persönlichen Zugriffstoken (Gitea: *Einstellungen* -> *Anwendungen*) mit dem Recht, Pakete zu lesen bzw. zu schreiben [108], [109]. Das normale Passwort funktioniert nicht, wenn Zwei-Faktor-Authentifizierung aktiv ist [107], und sollte ohnehin nicht in Skripten stehen.

```

docker login gitea.example.com          # Benutzername + Token (Schlüsselbund)
docker tag notizen:dev gitea.example.com/team/notizen:1.4.0
docker push gitea.example.com/team/notizen:1.4.0
docker pull gitea.example.com/team/notizen:1.4.0
  
```

Nach dem ersten Push erscheint das Image unter *Pakete* beim Besitzer. Dort lässt es sich mit einem Repository verknüpfen. Die Sichtbarkeit wird vom Besitzer geerbt, für private Images muss also der Besitzer (Benutzer oder Organisation) privat sein [109]. Für den Server legt man einen eigenen Token **nur mit Leserecht** an. Kommt dieser Token abhanden, kann damit niemand manipulierte Images hochladen.

TIPP

Gitea kann alte Image-Versionen automatisch löschen: In den Einstellungen des Besitzers unter *Pakete* -> *Aufräumregeln* legst du etwa fest, dass von jedem Image nur die letzten 20 Versionen behalten werden, Tags, die auf den regulären Ausdruck `\d+\.\d+\.\d+` passen, aber nie gelöscht werden [110]. Ohne solche Regeln wächst die Registry mit jedem CI-Lauf.

11.2 Eine Tag-Strategie

Tags sind beweglich (Kapitel 1) [19]. Damit jederzeit klar ist, welcher Code in welchem Image steckt, braucht es ein festes Schema:

Tag	Verwendung
Vollständiger Commit-Hash	Jeder Build aus der CI. Der 40-stellige SHA vermeidet Kollisionen und ist direkt auf einen Commit zurückführbar. Der tatsächlich deployte Inhalt wird zusätzlich über den Image-Digest festgelegt.
Version, z.B. <code>1.4.0</code>	Releases. Entsteht aus einem Git-Tag <code>v1.4.0</code> . Wird nie überschrieben.
<code>1.4</code> , <code>1</code>	Bequeme Zeiger auf die neueste Patch- bzw. Minor-Version. Nur für Nutzer, die automatisch aktualisieren wollen.
<code>latest</code>	Für eigene Deployments meiden. Auf dem Server steht eine unveränderliche <code>repository@sha256:...</code> -Referenz, damit der getestete Inhalt eindeutig ist.

11.3 Docker Hub

Öffentliche Images von Docker Hub lassen sich ohne Anmeldung laden. Stand Oktober 2026 gilt ein Sechs-Stunden-Kontingent von 100 Pulls je IPv4-Adresse beziehungsweise IPv6-/64-Netz für anonyme Nutzung und 200 Pulls für ein authentifiziertes Personal-Konto; bezahlte Tarife sind im Rahmen der Fair-Use-Regeln unbegrenzt. Ein Multi-Plattform-Pull zählt pro geladener Architektur [111]. Auf gemeinsam genutzten Build-Servern ist das Kontingent schnell erreicht, und Builds schlagen mit HTTP 429 und der Meldung *You have reached your pull rate limit* fehl [111]. Abhilfe: Runner anmelden, Builds und Pulls nicht unnötig wiederholen, Caches kontrolliert verwenden und bevorzugt offizielle Images oder Images bekannter Herausgeber einsetzen [112].

12 Sicherheit

Container isolieren gut, aber nicht vollständig: Sie teilen sich den Kernel des Hosts [113], und die Standardeinstellungen sind auf Bequemlichkeit ausgelegt, nicht auf minimale Rechte. Dieses Kapitel sammelt die Maßnahmen, die mit wenig Aufwand viel bringen.

12.1 Der Docker-Socket ist Root

Wer Befehle an den Docker-Daemon schicken darf, kann einen Container starten, der das komplette Dateisystem des Hosts einbindet, und hat damit volle Kontrolle über den Rechner [17]:

```
docker run --rm -it -v /:/host alpine chroot /host sh    # Root-Shell auf dem Host
```

Daraus folgen drei Regeln. **Erstens:** Die Gruppe `docker` auf einem Server ist gleichbedeutend mit `sudo` und wird entsprechend sparsam vergeben [114]. **Zweitens:** `/var/run/docker.sock` wird nie in einen Container eingebunden, außer das Werkzeug braucht ihn zwingend und ist vertrauenswürdig (etwa ein CI-Runner auf einem eigenen Rechner, siehe Git-Handbuch Kapitel 14) [115]. **Drittens:** Der Daemon wird nie per TCP ohne TLS im Netz erreichbar gemacht [17].

Wer die Gefahr grundsätzlich verringern will, kann Docker im *Rootless-Modus* betreiben: Daemon und Container laufen dann als normaler Benutzer [116]. Das hat Einschränkungen (etwa bei Ports unter 1024 und beim Netzwerk), ist für viele Anwendungen aber gut nutzbar [117].

12.2 Minimale Rechte im Container

`compose.prod.yaml` (Auszug)

```
api:
  image: gitea.example.com/team/notizen:1.4.0
  user: "10001:10001"                # nicht als root (steht meist schon im Image)
  read_only: true                    # Dateisystem des Containers schreibgeschützt
  tmpfs:
    - /tmp                          # beschreibbar, aber nur im Arbeitsspeicher
  cap_drop: [ALL]                   # alle Kernel-Sonderrechte entziehen
  security_opt:
    - no-new-privileges:true        # keine Rechteauserweiterung über setuid-Programme
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 512M                # bei Überschreitung: OOM-Kill, meist Exit-Code 137
        pids: 200                    # höchstens 200 Prozesse im Container
```

Maßnahme	Schutzwirkung
Nicht-Root-Benutzer	Eine Lücke in der Anwendung führt nicht zu Root-Rechten im Container, ein Ausbruch aus dem Container ist deutlich schwerer [115].
<code>read_only</code>	Ein Angreifer kann keine Programme im Container ablegen oder Dateien der Anwendung verändern [113].
<code>cap_drop: [ALL]</code>	Root im Container verliert Sonderrechte wie Netzwerkkonfiguration oder das Ändern von Dateibesitzern. Einzelne Rechte lassen sich mit <code>cap_add</code> gezielt zurückgeben [115].
<code>no-new-privileges</code>	Verhindert, dass ein Prozess über setuid-Programme mehr Rechte bekommt [115].
Ressourcenlimits	Ein Speicherleck oder eine Endlosschleife legt nicht den ganzen Server lahm [118].

Nicht jede Anwendung verträgt alle Maßnahmen. Datenbank-Images wie das offizielle PostgreSQL-Image wechseln beim Start selbst den Benutzer und brauchen dafür Rechte [119]. Das Vorgehen ist deshalb: alles setzen, starten, Logs lesen und nur zurücknehmen, was nachweislich gebraucht wird.

12.3 Geheimnisse

Umgebungsvariablen sind der übliche Weg, Container zu konfigurieren, für Passwörter aber nicht ideal: Sie erscheinen in `docker inspect`, werden an Kindprozesse vererbt und landen in Fehlerberichten [120]. Besser sind Dateien, die nur im Container sichtbar sind. Compose bietet dafür `secrets`:

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_passwort # offizielle Images kennen *_FILE
    secrets: [db_passwort]
  api:
    image: gitea.example.com/team/notizen:1.4.0
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_passwort
    secrets: [db_passwort]

secrets:
  db_passwort:
    file: ./secrets/db_passwort.txt # nur auf dem Server, nie im Git
```

Compose stellt die Datei im Container unter `/run/secrets/db_passwort` bereit [120]. Bei einer lokalen `file:`-Quelle ist das technisch ein schreibgeschützter Bind Mount - kein verschlüsselter Secret-Speicher [69]. Viele offizielle Images (PostgreSQL, MariaDB) verstehen die Konvention `VARIABLE_FILE` bereits [43], [121]. Die eigene Anwendung muss das Passwort selbst aus der Datei lesen:

`app/db.py` (Auszug)

```
import os
from pathlib import Path

def db_passwort() -> str:
    if datei := os.environ.get("DB_PASSWORD_FILE"):
        return Path(datei).read_text().strip()
    return os.environ["DB_PASSWORD"] # Fallback für die Entwicklung
```

Bei file-basierten Compose-Secrets werden die optionalen Angaben `uid`, `gid` und `mode` nicht umgesetzt; Besitzer und Modus der Host-Datei gelten auch im Container [69]. Im Beispiel läuft die API als UID 10001. Die Datei wird deshalb rootgeschützt angelegt und gezielt für diese UID lesbar gemacht:

```
sudo install -d -o root -g root -m 0711 /srv/notizen/secrets
openssl rand -base64 32 | sudo tee /srv/notizen/secrets/db_passwort.txt >/dev/null
sudo chown 10001:10001 /srv/notizen/secrets/db_passwort.txt
sudo chmod 0400 /srv/notizen/secrets/db_passwort.txt
docker compose run --rm api test -r /run/secrets/db_passwort
```

Das PostgreSQL-Image liest sein `_FILE`-Secret während der Initialisierung mit Root-Rechten [119]. Nutzen mehrere Nicht-Root-Dienste dasselbe Geheimnis, braucht es getrennte Dateien, eine gemeinsame numerische Gruppe oder passende ACLs. Die Dateien gehören nie in die Versionsverwaltung.

12.4 Images auf Schwachstellen prüfen

Basis-Images und Python-Pakete enthalten regelmäßig bekannte Sicherheitslücken [113]. Ein Scanner vergleicht den Inhalt eines Images mit Schwachstellen-Datenbanken [122]:

```
brew install trivy
trivy image notizen:1.4.0 # alle Befunde
```

```
trivy image --severity HIGH,CRITICAL notizen:1.4.0
trivy image --exit-code 1 --exit-on-eol 1 --severity CRITICAL notizen:1.4.0
```

Nicht behobene Lücken sind nicht automatisch irrelevant: Man kann das betroffene Paket entfernen, das Basis-Image wechseln, die Funktion abschalten oder das Risiko vorübergehend dokumentieren. Die CI blockiert im Beispiel behebbare und nicht behebbare kritische Findings; begründete Ausnahmen werden eng befristet und nachvollziehbar gepflegt. `--exit-on-eol 1` beendet Trivy außerdem mit Exit-Code 1 und verhindert so, dass ein nicht mehr unterstütztes Betriebssystem unbemerkt weiterläuft [123]. Die wirksamste Routine bleibt: **regelmäßig neu bauen und erneut prüfen** [115].

12.5 Herkunft und Integrität von Images

Ein Digest schützt davor, dass sich ein bereits ausgewählter Inhalt verändert, sagt aber noch nicht, wer ihn gebaut hat [19]. Docker Content Trust wurde mit Engine 29 aus der Docker-CLI entfernt [4]. Für signierte Images verwendet man heute Werkzeuge wie *Cosign* oder *Notation* und prüft die Signatur vor dem Deployment [124], [125]. Die CI erzeugt zusätzlich SBOM und Provenance (Kapitel 14) [126]. Signaturen, Scanner und SBOM ergänzen einander; keines davon ersetzt Tests.

12.6 Checkliste

Maßnahme
☐ Images aus vertrauenswürdigen Quellen, mit fester Version [113], [115]
☐ Multi-Stage-Build, keine Build-Werkzeuge im Laufzeit-Image (Kapitel 8)
☐ Anwendung läuft als Nicht-Root-Benutzer
☐ <code>read_only</code> , <code>cap_drop: [ALL]</code> , <code>no-new-privileges</code> , Ressourcenlimits
☐ Passwörter als Secrets-Dateien, <code>.env</code> und <code>secrets/</code> nicht im Git
☐ Nur der Reverse Proxy veröffentlicht Ports, alles andere an <code>127.0.0.1</code> oder gar nicht (Kapitel 6)
☐ Docker-Socket in keinem Anwendungscontainer, Gruppe <code>docker</code> nur für Administratoren
☐ Regelmäßig neu bauen und mit Trivy prüfen, idealerweise automatisch in der CI
☐ Build-Geheimnisse nur über BuildKit-Secret- oder SSH-Mounts, nie über <code>ARG</code> , <code>ENV</code> oder <code>COPY</code>
☐ SBOM, Provenance und bei erhöhtem Schutzbedarf Image-Signaturen prüfen
☐ Host regelmäßig patchen, cgroup v2 sowie seccomp/AppArmor nicht ohne Grund deaktivieren [4], [127]

13 Betrieb auf dem Server

Am Ende soll die Anwendung auf einem Linux-Server laufen: erreichbar über HTTPS, mit gesicherten Daten, sauberen Logs und einem einfachen Weg zu Updates und Rollbacks. Für eine Anwendung dieser Größe genügt ein einzelner Server mit Docker Compose vollkommen.

13.1 Docker Engine installieren

Auf dem Server wird kein Docker Desktop gebraucht, sondern die Docker Engine aus dem offiziellen Paket-Repository von Docker. Die Pakete der Distribution (`docker.io`) sind inoffiziell und können mit den offiziellen Paketen kollidieren, das Komfortskript `get.docker.com` ist für Testrechner gedacht [128]. Für Debian sieht die Installation so aus (Ubuntu analog, aktuelle Fassung auf docs.docker.com prüfen):

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $VERSION_CODENAME
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin docker-ce-rootless-extras
sudo docker run --rm hello-world
```

Docker Engine 29 hat cgroup v1 als veraltet markiert und unterstützt es noch bis mindestens Mai 2029 [129]. Auf einem neuen Server zeigt `docker info --format '{{.CgroupVersion}}'` deshalb `2`. Bestehende Systeme mit cgroup v1 werden vor einer späteren Entfernung des Supports auf cgroup v2 migriert; für Engine 29 ist noch keine sofortige Abschaltung nötig.

Grundkonfiguration des Daemons

`/etc/docker/daemon.json`

```
{
  "log-driver": "local",
  "log-opts": { "max-size": "20m", "max-file": "5" },
  "live-restore": true
}
```

```
sudo systemctl restart docker
```

Ohne Begrenzung schreibt Docker die Ausgaben jedes Containers unbegrenzt auf die Platte, bis sie voll ist [130]. Der Treiber `local` rotiert und komprimiert die Logs, hier auf höchstens fünf Dateien mit je 20 MB pro Container [131]. `live-restore` kann Container während eines Daemon-Neustarts oder kompatiblen Patch-Updates weiterlaufen lassen. Bei einem Major-Upgrade oder Änderungen an Storage-Treiber, Bridge-Netz oder anderen Daemon-Optionen gilt diese Garantie nicht [132]. Die Einstellungen gelten nur für neu erzeugte Container [130].

13.2 Aufbau auf dem Server

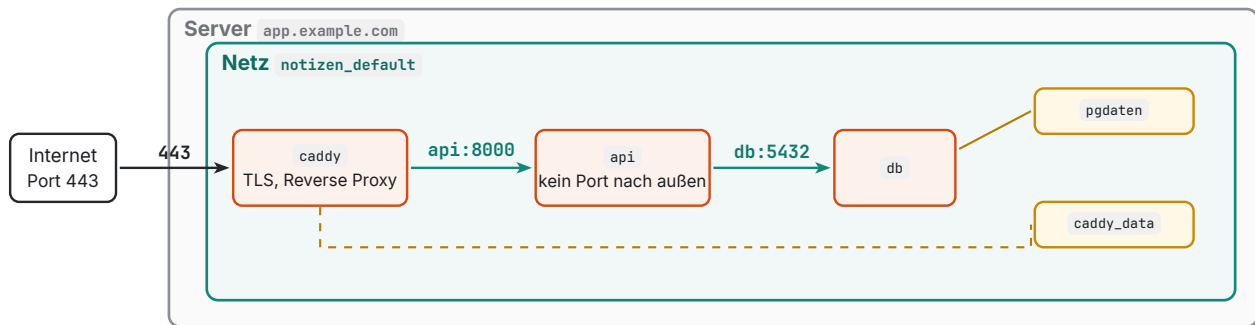


Abb. 15: Nur Caddy ist von außen erreichbar. API und Datenbank sprechen im internen Netz miteinander.

Das Projekt liegt in einem eigenen Verzeichnis, typischerweise unter `/srv`:

Verzeichnisstruktur auf dem Server

```

/srv/notizen/
├── compose.yaml      # Kopie von compose.prod.yaml aus dem Repository
├── .env              # IMAGE_REF=...@sha256:... und weitere, nicht geheime Werte
├── Caddyfile
├── secrets/
│   └── db_passwort.txt # Besitzer 10001:10001, Modus 0400
├── deploy.sh         # wird von der CI per SSH aufgerufen (Kapitel 14)
└── backups/

```

13.3 Reverse Proxy mit automatischem HTTPS

Ein Reverse Proxy nimmt alle Anfragen auf Port 80 und 443 entgegen, kümmert sich um die TLS-Zertifikate und leitet an die Anwendung weiter. **Caddy** ist dafür besonders einfach: Er holt und erneuert Let's-Encrypt-Zertifikate selbstständig, sobald der DNS-Eintrag auf den Server zeigt [133]. Die komplette Konfiguration für eine Domain:

Caddyfile

```

app.example.com {
    reverse_proxy api:8000
    encode zstd gzip
}

```

13.4 Die Produktions-Compose-Datei

/srv/notizen/compose.yaml

```

name: notizen

services:
  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data # Zertifikate: unbedingt persistent!
      - caddy_config:/config
    restart: unless-stopped

  api:
    image: ${IMAGE_REF:?IMAGE_REF fehlt}
    environment:
      DB_HOST: db
      DB_PASSWORD_FILE: /run/secrets/db_passwort
    secrets: [db_passwort]

```

```

read_only: true
tmpfs: [/tmp]
cap_drop: [ALL]
security_opt: ["no-new-privileges:true"]
depends_on:
  db:
    condition: service_healthy
restart: unless-stopped

db:
  image: postgres:18
  environment:
    POSTGRES_USER: notizen
    POSTGRES_DB: notizen
    POSTGRES_PASSWORD_FILE: /run/secrets/db_password
  secrets: [db_password]
  volumes:
    - pgdaten:/var/lib/postgresql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U notizen -d notizen"]
    interval: 10s
    retries: 10
    restart: unless-stopped

secrets:
  db_password:
    file: ./secrets/db_password.txt

volumes:
  pgdaten:
  caddy_data:
  caddy_config:

```

Die freigegebene Image-Referenz steht nicht in der Datei, sondern kommt über `IMAGE_REF` aus der `.env`, zum Beispiel `gitea.example.com/team/notizen@sha256:...`. Der Digest bezeichnet unveränderlich exakt das in der CI geprüfte Image. Ein Update oder Applikations-Rollback ändert damit genau eine Zeile. Das `?:` sorgt dafür, dass Compose bei fehlendem Wert abbricht.

Die Tags `caddy:2` und `postgres:18` erhalten Sicherheitsupdates und sind deshalb beweglich [43]. Sie werden nicht ungeprüft bei jedem API-Deployment aktualisiert, sondern in einem eigenen Wartungsfenster gezogen, getestet und bei streng reproduzierbaren Umgebungen zusätzlich über einen dokumentierten Digest festgelegt. Für das selbst gebaute API-Image wird nach der CI der konkrete Registry-Digest protokolliert.

Das file-basierte Secret muss für den Anwendungsbenutzer lesbar sein; `uid`, `gid` und `mode` im Compose-Eintrag würden bei `file:` nicht helfen [69]. Die Einrichtung aus Kapitel 12 setzt deshalb den numerischen Besitzer auf UID 10001. Vor dem ersten Start prüfen:

```

docker compose run --rm api test -r /run/secrets/db_password
docker compose run --rm db test -r /run/secrets/db_password

```

13.5 Updates und Rollbacks

```

cd /srv/notizen
sed -i 's|^IMAGE_REF=.*|IMAGE_REF=gitea.example.com/team/notizen@sha256:...|' .env
docker compose pull api # nur das neue API-Image laden
docker compose up -d --wait --wait-timeout 60 api # API umstellen und auf Health warten
docker compose ps && docker compose logs --tail 30 api

```

Alte API-Images bleiben zunächst für einen schnellen Rollback lokal vorhanden und werden erst nach der festgelegten Aufbewahrungsfrist durch eine getrennte Wartungsaufgabe entfernt.

Ein Applikations-Rollback ist derselbe Ablauf mit der vorherigen Version, solange die Datenbank dazu kompatibel bleibt. Datenbankänderungen werden deshalb nach dem Expand/Contract-Muster gebaut: zunächst nur additive, rückwärtskompatible Änderungen; Entfernen alter Spalten erst in einem späteren Release [134]. Ein Dump ist die Notfallabsicherung, aber kein schneller automatischer Rollback einer laufenden Migration. In der Praxis erledigt diese Schritte die CI (Kapitel 14), von Hand macht man sie nur im Notfall.

ACHTUNG

Das früher beliebte Werkzeug *Watchtower*, das laufende Container automatisch auf neue Images aktualisiert, wurde im Dezember 2025 archiviert und wird nicht mehr gepflegt [135]. Automatische Updates ohne Tests und ohne Backup sind für eigene Anwendungen ohnehin riskant. Besser: Deployments über die CI, und für Basis-Images wie `postgres` oder `caddy` nur **benachrichtigen** lassen, etwa mit dem Werkzeug *Diun*, und bewusst aktualisieren [136].

13.6 Backups

Ein Skript sichert die Datenbank täglich und behält die Sicherungen der letzten 14 Tage:

`/srv/notizen/backup.sh`

```
#!/usr/bin/env bash
set -euo pipefail
umask 077
cd /srv/notizen
mkdir -p backups
exec 9>backups/.backup.lock
flock -n 9 || { echo "Backup läuft bereits" >&2; exit 1; }
zeit="$(date -u +%Y%m%dT%H%M%S)"
tmp="backups/.notizen-$zeit.dump.tmp"
datei="backups/notizen-$zeit.dump"
trap 'rm -f "$tmp"' EXIT
docker compose exec -T db pg_dump -U notizen -Fc notizen > "$tmp"
docker compose exec -T db pg_restore -l < "$tmp" >/dev/null
mv "$tmp" "$datei"
trap - EXIT
find backups/ -name 'notizen-*.dump' -mtime +14 -delete
```

```
chmod 700 /srv/notizen/backup.sh
crontab -e
# Eintrag: täglich um 3:15 Uhr
15 3 * * * /srv/notizen/backup.sh >> /srv/notizen/backups/backup.log 2>&1
```

Das `-T` bei `exec` schaltet die Pseudo-Terminal-Zuweisung ab, die sonst die Binärdaten des Dumps verfälschen kann [137]. Zeit bis auf die Sekunde verhindert, dass zwei Deployments am selben Tag dieselbe Datei überschreiben; temporäre Datei und `pg_restore -l` verhindern, dass ein unvollständiger Dump wie ein gültiges Backup aussieht [138], [139]. Rollen und Tablespace werden bei Bedarf zusätzlich mit `pg_dumpall --globals-only` gesichert. Ein Backup auf demselben Server schützt nur vor Bedienfehlern, nicht vor dessen Ausfall. Die Dumps gehören deshalb verschlüsselt an einen zweiten Ort, etwa mit `restic` auf eine NAS oder in Object Storage [140]. Der Backup-Job wird überwacht, und mindestens quartalsweise wird ein Dump in eine frische Datenbank eingespielt.

13.7 Beobachten

Befehl	Zweck
<code>docker compose ps</code>	Laufen alle Dienste, sind sie <i>healthy</i> ?
<code>docker compose logs --since 1h api</code>	Ausgaben der letzten Stunde
<code>docker stats --no-stream</code>	aktueller Verbrauch von CPU und Speicher pro Container
<code>docker system df</code>	Platzverbrauch von Images, Volumes, Build-Cache
<code>df -h /var/lib/docker</code>	freier Platz auf der Partition mit den Docker-Daten

Für mehr als einen Blick von Hand lohnt sich ein externer Uptime-Monitor, der die Health-Route der Anwendung regelmäßig von außen aufruft und bei Ausfall benachrichtigt (etwa *Uptime Kuma*, selbst als Container auf einem anderen Rechner betrieben).

AUS DER PRAXIS

Synology: Der *Container Manager* versteht Compose-Dateien als *Projekte* [141], doch Engine-/Compose-Version, CPU-Architektur, Dateirechte und unterstützte Felder hängen vom DSM-Modell und Release ab. Deshalb `docker version`, `docker compose version` und `docker compose config` vorab prüfen. Der Weg über SSH und `docker compose` ist gut dokumentierbar; Bind Mounts legt man bevorzugt unter `/volume1/docker/<projekt>` an und bezieht sie ausdrücklich in das NAS-Backup ein.

14 Bauen und Ausliefern mit Gitea Actions

Das Git-Handbuch beschreibt in den Kapiteln 13 bis 15 Gitea Actions, Runner, Secrets und das SSH-Deployment mit eingeschränktem Schlüssel ausführlich. Dieses Kapitel setzt darauf auf und zeigt die Docker-spezifischen Teile: Tests gegen eine echte Datenbank, einen reproduzierbaren Build mit SBOM und Provenance, den Test des fertigen Images sowie ein abgesichertes Deployment mit Backup und Applikations-Rollback.

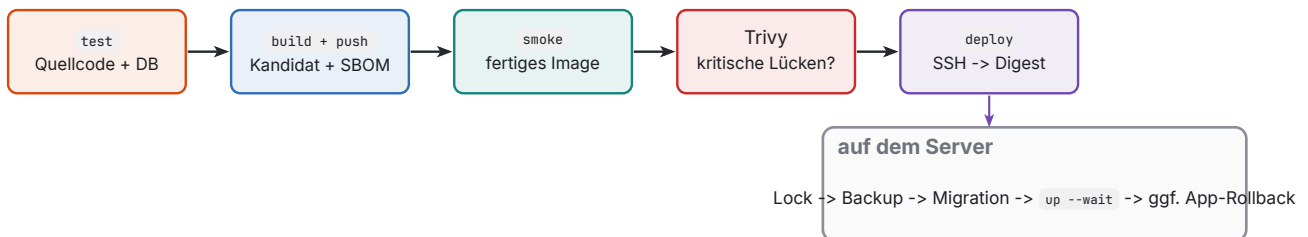


Abb. 16: Nur der nach dem Build getestete und gescannte Digest wird deployt.

14.1 Vertrauenswürdige Werkzeuge im Runner

Installationsbefehle wie `curl ... | sh` sparen Zeilen, führen aber bei jedem Lauf ungeprüften, veränderlichen Code aus. Der dedizierte Runner dieses Beispiels stellt Docker Buildx, `uv`, Trivy und `curl` in festgelegten Versionen bereit. Das Runner-Image selbst wird separat aktualisiert, per Checksum oder Signatur geprüft und getestet. Auch Actions werden auf den vollständigen Commit-SHA statt nur auf einen beweglichen Tag gepinnt [142].

ACHTUNG

Ein Runner mit Zugriff auf `/var/run/docker.sock` hat praktisch Root-Rechte auf seinem Host [17]. Er darf deshalb nur vertrauenswürdige Workflows ausführen, ist dediziert oder kurzlebig und erhält bei Pull Requests aus fremden Forks weder Secrets noch Docker-Socket-Zugriff [143], [144].

14.2 Tests gegen eine echte Datenbank

Gitea Actions kann für einen Job, dessen Runner-Label auf ein Docker-Image zeigt, Hilfscontainer starten (*services*). Der Job erreicht sie über ihren Namen, genau wie in einem Compose-Netz [145]. So laufen die Tests gegen dieselbe PostgreSQL-Hauptversion wie in Produktion statt gegen eine Attrappe:

`.gitea/workflows/ci.yml (Auszug)`

```

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      db:
        image: postgres:18
        env:
          POSTGRES_USER: notizen
          POSTGRES_PASSWORD: test
          POSTGRES_DB: notizen
        options: >-
          --health-cmd "pg_isready -U notizen"
          --health-interval 5s
          --health-retries 10
  
```

```
env:
  DATABASE_URL: postgresql://notizen:test@db:5432/notizen
steps:
  # actions/checkout v4.2.2; SHA statt beweglichem @v4
  - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
  - run: uv --version
  - run: uv sync --locked
  - run: uv run pytest
```

Der Runner wartet, bis der Healthcheck des Service-Containers erfolgreich ist, bevor er die Schritte startet [145]. Das prüft den Quellcode. Zusätzlich wird im nächsten Job das wirklich veröffentlichte Image als Container gestartet.

14.3 Image bauen, attestieren und als Kandidat veröffentlichen

BuildKit kann neben dem Image eine Software-Stückliste (SBOM) und Provenance über den Build erzeugen [126]. Attestations werden zuverlässig in einer Registry gespeichert; deshalb erhält der Kandidat zunächst nur den vollständigen Commit-SHA als internen Tag. Erst nach Smoke-Test und Scan darf sein Digest deployt oder unter einem Release-Tag beworben werden.

`.gitea/workflows/deploy.yml` (Auszug)

```
build:
  needs: [test]
  runs-on: ubuntu-latest
  outputs:
    digest: ${ steps.image.outputs.digest }
  env:
    IMAGE: gitea.example.com/team/notizen
  steps:
    - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
    - name: An Registry anmelden
      env:
        TOKEN: ${ secrets.REGISTRY_TOKEN }
      run: |
        echo "$TOKEN" | docker login gitea.example.com \
          -u "${ gitea.actor }" --password-stdin
    - name: Kandidat mit SBOM und Provenance bauen
      run: |
        docker buildx build --pull --platform linux/amd64 \
          --sbom=true --provenance=mode=max \
          --label org.opencontainers.image.revision="$GITHUB_SHA" \
          -t "$IMAGE:$GITHUB_SHA" --push .
    - name: Unveränderliche Referenz bestimmen
      id: image
      run: |
        docker pull "$IMAGE:$GITHUB_SHA"
        REF="$(docker image inspect "$IMAGE:$GITHUB_SHA" \
          --format '{{index .RepoDigests 0}}')"
        DIGEST="${REF##*@}"
        [[ "$DIGEST" =~ ^sha256:[0-9a-f]{64}$ ]]
        echo "digest=$DIGEST" >> "$GITHUB_OUTPUT"
        echo "Prüfkandidat: $REF"
    - name: Fertiges Image gegen PostgreSQL starten
      run: |
        REF="$IMAGE@${ steps.image.outputs.digest }"
        trap 'docker rm -f ci-api ci-db 2>/dev/null || true; \
          docker network rm ci-notizen 2>/dev/null || true' EXIT
        docker network create ci-notizen
        docker run -d --name ci-db --network ci-notizen \
          -e POSTGRES_USER=notizen -e POSTGRES_PASSWORD=test \
          -e POSTGRES_DB=notizen postgres:18
        for i in $(seq 1 30); do
          docker exec ci-db pg_isready -U notizen && break
```

```

    [[ "$i" == 30 ]] && exit 1; sleep 2
done
docker run --rm --network ci-notizen \
  -e DB_HOST=ci-db -e DB_PASSWORD=test "$REF" \
  alembic upgrade head
docker run -d --name ci-api --network ci-notizen \
  -p 127.0.0.1:18000:8000 \
  -e DB_HOST=ci-db -e DB_PASSWORD=test "$REF"
for i in $(seq 1 30); do
  curl -fsS http://127.0.0.1:18000/health && break
  [[ "$i" == 30 ]] && { docker logs ci-api; exit 1; }; sleep 2
done
- name: Kandidaten scannen
  run: |
    trivy image --exit-code 1 --exit-on-eol 1 --severity CRITICAL \
      "$IMAGE@${ steps.image.outputs.digest }"

```

- `--pull` nimmt Sicherheitsupdates des beweglichen Basis-Tags in einen neuen Kandidaten auf [105].
- `--sbom=true` dokumentiert enthaltene Pakete; `--provenance=mode=max` dokumentiert Build-Quelle und Parameter, legt dabei aber auch Build-Argument-Werte offen (deshalb keine Geheimnisse in `--build-arg`) [146].
- Der Smoke-Test und Trivy verwenden `repository@sha256:...`, also exakt den veröffentlichten Inhalt [147]. Ein fehlgeschlagener Kandidat bleibt zwar für die Analyse in der Registry, wird aber weder promoted noch deployt.
- Der Registry-Token ist ein persönlicher Zugriffstoken mit Schreibrecht für Pakete, weil der automatische `GITEA_TOKEN` Pakete nicht hochladen kann [148].

Für besonders schützenswerte Lieferketten signiert die CI den geprüften Digest anschließend mit Cosign oder Notation [149]. Docker Content Trust gehört seit Engine 29 nicht mehr zur Docker-CLI [4]; sein Notary-v1-Dienst wird am 8. Dezember 2026 abgeschaltet [6]. Die Deployment-Seite prüft dann Signatur und erlaubte Identität, bevor sie den Digest übernimmt.

14.4 Das Deployment-Skript für Compose

Der Deploy-Job ruft per SSH mit einem eingeschränkten Schlüssel (`restrict,command=...`) [150] das folgende Skript auf und übergibt ausschließlich den geprüften Digest. Einrichtung von Schlüssel, `authorized_keys` und `known_hosts` siehe Git-Handbuch Kapitel 14.

/srv/notizen/deploy.sh

```

#!/usr/bin/env bash
set -euo pipefail
umask 077

DIGEST="${SSH_ORIGINAL_COMMAND:-}"
if [[ ! "$DIGEST" =~ ^sha256:[0-9a-f]{64}$ ]]; then
  echo "Ungültiger Digest: '$DIGEST'" >&2; exit 1
fi

cd /srv/notizen
exec 9>/run/lock/notizen-deploy.lock
flock -n 9 || { echo "Deployment läuft bereits" >&2; exit 1; }

NEU="gitea.example.com/team/notizen@$DIGEST"
NEXT_ENV="$(mktemp /srv/notizen/.env.next.XXXXXX)"
trap 'rm -f "$NEXT_ENV"' EXIT
cp .env "$NEXT_ENV"
if grep -q '^IMAGE_REF=' "$NEXT_ENV"; then
  sed -i "s|^IMAGE_REF=.*|IMAGE_REF=$NEU|" "$NEXT_ENV"
else
  printf 'IMAGE_REF=%s\n' "$NEU" >> "$NEXT_ENV"
fi

```

```

./backup.sh
docker compose --env-file "$NEXT_ENV" pull api
docker compose --env-file "$NEXT_ENV" run --rm api alembic upgrade head

if docker compose --env-file "$NEXT_ENV" up -d --wait --wait-timeout 60 api; then
    mv "$NEXT_ENV" .env
    trap - EXIT
    echo "Deployed $NEU"
    exit 0
fi

echo "Neue API nicht gesund; stelle vorherige App-Version wieder her" >&2
docker compose --env-file .env pull api || true
docker compose --env-file .env up -d --wait --wait-timeout 60 api || true
docker compose logs --tail 80 api >&2
exit 1

```

Das Lock verhindert parallele Deployments. Die produktive `.env` wird erst nach einem erfolgreichen Healthcheck ersetzt. Bei einem Fehler wird die vorherige **Applikationsversion** wieder gestartet. Die bereits ausgeführte Datenbankmigration wird absichtlich nicht automatisch zurückgerollt: Migrationen müssen nach dem Expand/Contract-Muster rückwärtskompatibel sein [134]. Ein Datenbank-Restore ist ein bewusster Notfallvorgang mit möglichem Datenverlust seit dem Backup.

Alte Images werden nicht im selben Skript sofort gelöscht, damit ein Rollback verfügbar bleibt. Eine getrennte, überwachte Wartungsaufgabe entfernt sie nach der festgelegten Aufbewahrungsfrist.

MERKE

Das Zusammenspiel der beiden Handbücher: Git regelt, **welcher Code** ausgeliefert wird. Die CI baut einen Kandidaten, versieht ihn mit SBOM und Provenance und prüft exakt seinen Digest. Docker Compose startet genau diesen unveränderlichen Digest mit Daten, Geheimnissen und Netzen. Zurückdrehen lässt sich die Anwendung; Datenbankänderungen brauchen zusätzlich ein bewusstes Migrations- und Restore-Konzept.

15 Compose, Swarm oder Kubernetes?

Docker Compose startet alle Container auf **einem** Rechner. Fällt dieser Rechner aus, steht die Anwendung. Werkzeuge zur *Orchestrierung* verteilen Container dagegen auf mehrere Rechner, starten sie bei Ausfällen anderswo neu und aktualisieren ohne Unterbrechung. Die Frage ist, ab wann sich das lohnt.

15.1 Ist Docker Swarm tot?

Nicht tot, aber ein Nischenprodukt. Der Open-Source-*Swarm-Modus* ist weiterhin in Docker Engine eingebaut (`docker swarm init`) [151] und erhält Korrekturen [4]. Seine Featureentwicklung und sein Ökosystem sind jedoch deutlich langsamer geworden [6]; beispielsweise lässt sich das neue, noch experimentelle *nftables*-Firewall-Backend von Docker Engine 29 auf Swarm-Knoten noch nicht aktivieren [152]. Davon zu unterscheiden ist das kommerzielle Produkt Mirantis Kubernetes Engine: Laut Mirantis ist MKE 4 Kubernetes-basiert und unterstützt den früheren Swarm-Modus von MKE 3 nicht mehr [153].

Für bestehende Installationen ist Swarm weiterhin eine solide, einfache Lösung. Für einen **Neueinstieg** in die Orchestrierung ist es 2026 aber kaum noch zu empfehlen, weil das Wissen und die Werkzeuge rund um Kubernetes deutlich breiter und zukunftssicherer sind.

15.2 Die Optionen im Vergleich

	Compose	Swarm	Kubernetes (z.B. k3s) [154]
Rechner	einer	mehrere	mehrere
Ausfallsicherheit	nein, Neustart nach Reboot	ja	ja
Updates ohne Unterbrechung	nein (Sekunden)	ja (rolling update) [155]	ja [156]
Lernaufwand	gering	gering	hoch
Betriebsaufwand	minimal	gering	spürbar, auch mit k3s
Konfiguration	<code>compose.yaml</code>	fast dieselbe Datei	eigene YAML-Manifeste, Helm-Charts
Zukunft	Kernwerkzeug von Docker	weiter verfügbar, langsame Entwicklung	Industriestandard

Dazwischen gibt es Werkzeuge, die Deployments auf einzelne oder wenige Server ohne Orchestrierung komfortabler machen, etwa *Kamal*, das Container per SSH ausrollt und dabei Updates ohne Unterbrechung über einen eigenen Proxy ermöglicht [157], [158].

15.3 Empfehlung

- **Eine Anwendung, ein bis zwei Server, kleines Team:** Compose, wie in diesem Buch beschrieben. Ein Update unterbricht die API für wenige Sekunden, ein Serverausfall wird durch Backups und ein vorbereitetes Wiederaufsetzen auf einem Ersatzrechner abgefangen (die Compose-Datei, `.env` und die Secrets genügen dafür).
- **Echte Hochverfügbarkeit gefordert** (vertraglich zugesicherte Verfügbarkeit, mehrere Instanzen hinter einem Load Balancer): direkt Kubernetes, für kleine Umgebungen die leichtgewichtige Distribution *k3s* [154]. Images, Dockerfiles, Registries und viele Sicherheitsprinzipien bleiben nützlich. Neu gelernt werden müssen unter anderem Netzwerkmodell, Zustandsverwaltung, Identitäten, Secrets, Health- und Readiness-Semantik, Ressourcenplanung, Rollouts und Observability.

- **Swarm** nur, wenn es bereits im Einsatz ist.

A Befehlsübersicht

Vollständige Optionen: Referenz der Docker-CLI [159].

COLIMA UND KONTEXTE

<code>colima start / stop / status</code>	VM starten, stoppen, Zustand
<code>colima start --edit</code>	Konfiguration ändern
<code>brew services start colima</code>	Autostart bei Anmeldung
<code>docker context ls / use colima</code>	Kontexte anzeigen / wechseln

CONTAINER

<code>docker run -d --name x -p 127.0.0.1:8000:8000 img</code>	im Hintergrund starten
<code>docker run --rm -it img sh</code>	Wegwerf-Container mit Shell
<code>docker ps -a</code>	alle Container
<code>docker logs -f --tail 100 x</code>	Ausgaben verfolgen
<code>docker exec -it x sh</code>	Shell im laufenden Container
<code>docker stop / start / rm x</code>	anhalten / fortsetzen / entfernen
<code>docker inspect x</code>	Details als JSON
<code>docker stats</code>	Ressourcenverbrauch live
<code>docker cp x:/pfad .</code>	Dateien herauskopieren

IMAGES

<code>docker build -t name:tag .</code>	Image bauen
<code>docker build --no-cache --pull ...</code>	komplett neu, Basis frisch
<code>docker image ls / history img</code>	Images / Schichten
<code>docker tag a b</code>	zweiten Namen vergeben
<code>docker push / pull name:tag</code>	hoch- / herunterladen
<code>docker login gitea.example.com</code>	an Registry anmelden

VOLUMES UND NETZE

<code>docker volume ls / inspect v</code>	Volumes anzeigen
<code>docker volume rm v</code>	Volume löschen (!)
<code>docker network create n</code>	Netz mit Namensauflösung
<code>docker network inspect n</code>	Teilnehmer und IPs
<code>docker run --rm -it --network n nicolaka/netshoot</code>	Netzwerk-Werkzeuge

MEHRERE PLATTFORMEN

<code>docker build --platform linux/amd64 ...</code>	für x86 bauen
<code>docker buildx create --name multi --driver docker-container --use</code>	Builder anlegen
<code>docker buildx build --platform linux/amd64,linux/arm64 --push ...</code>	Multi-Plattform-Image
<code>docker buildx imagetools inspect img</code>	Varianten prüfen

COMPOSE

<code>docker compose up -d</code>	Sollzustand herstellen
<code>docker compose up -d --build</code>	Build anstoßen, Cache erlaubt
<code>docker compose up -d --wait</code>	starten und auf Health warten
<code>docker compose up --watch</code>	Entwicklung mit Sync
<code>docker compose ps / logs -f api</code>	Status / Logs
<code>docker compose exec db psql -U notizen</code>	in laufendem Dienst
<code>docker compose run --rm api pytest</code>	Einmal-Befehl
<code>docker compose pull</code>	neue Images laden
<code>docker compose config</code>	aufgelöste Konfiguration
<code>docker compose down</code>	entfernen, Daten bleiben
<code>docker compose down -v</code>	inkl. Volumes (!)
<code>docker compose --profile werkzeuge up -d</code>	optionale Dienste

FEHLERSUCHE

<code>docker compose run --rm --entrypoint sh api</code>	Shell trotz Absturz
<code>docker inspect -f '{{.State.ExitCode}}' x</code>	Exit-Code
<code>docker inspect -f '{{json .State.Health}}' x</code>	Healthcheck-Verlauf
<code>docker events --since 10m</code>	Ereignisse des Daemons

AUFRÄUMEN UND SICHERUNG

<code>docker system df</code>	Platzverbrauch
<code>docker system prune</code>	Ungenutztes löschen (ohne Volumes)
<code>docker volume prune -a</code>	alle ungenutzten Volumes (!)
<code>docker image prune -a</code>	alle ungenutzten Images
<code>docker exec db pg_dump -U u -Fc db > x.dump</code>	Datenbank sichern
<code>trivy image name@sha256:...</code>	exakten Digest prüfen

B Glossar

Basis-Image

Image, auf dem ein Dockerfile mit `FROM` aufbaut, z.B.

`python:3.14-slim`.

Bind Mount

Direkt eingeblendetes Verzeichnis des Hosts im Container.

Build-Kontext

Verzeichnis, dessen Inhalt beim Bauen für `COPY` erreichbar ist, gefiltert durch `.dockerignore`.

BuildKit / Buildx

Aktuelles Build-System von Docker mit Cache-Mounts und Multi-Plattform-Builds.

Attestation

Mit einem Image veröffentlichte, überprüfbare Metadaten wie SBOM oder Build-Provenance.

cgroups

Kernel-Mechanismus zur Begrenzung von CPU, Speicher und Prozessen.

Colima

Kommandozeilenwerkzeug, das am Mac eine Linux-VM mit Docker-Daemon verwaltet.

Compose

Werkzeug, das eine Anwendung aus mehreren Containern deklarativ in `compose.yaml` beschreibt.

Container

Abgeschotteter Linux-Prozess mit eigenem Dateisystem aus einem Image und eigener Schreibschicht.

Daemon (`dockerd`)

Hintergrunddienst, der Images, Container, Netze und Volumes verwaltet.

Digest

Unveränderliche Prüfsumme eines Images (`sha256:...`).

Dockerfile

Bauanleitung für ein Image.

Healthcheck

Regelmäßig ausgeführter Befehl, der meldet, ob ein Container funktioniert.

Image

Unveränderliche, geschichtete Vorlage für Container.

Image-Index

Liste von Image-Varianten für verschiedene Architekturen unter einem Tag.

Kontext (CLI)

Benannte Verbindung der CLI zu einem Daemon, z.B.

`colima`.

Layer / Schicht

Schreibgeschützte Dateisystemänderung eines Build-Schritts.

Multi-Stage-Build

Dockerfile mit mehreren Stufen, von denen nur die letzte zum Image wird.

Namespace

Kernel-Mechanismus, der einem Prozess eine eigene Sicht auf Prozesse, Netz und Dateisystem gibt.

Port-Veröffentlichung

Weiterleitung eines Host-Ports in einen Container (`-p`).

Profil (Compose)

Gruppe optionaler Dienste, die nur auf Anforderung starten.

Registry

Server zum Speichern und Verteilen von Images, z.B. Docker Hub oder Gitea.

Reverse Proxy

Vorgeschalteter Server, der Anfragen entgegennimmt, TLS erledigt und weiterleitet.

Rosetta

Apples Übersetzungsschicht, mit der x86-Container auf Apple Silicon laufen.

Schreibschicht

Beschreibbare oberste Schicht eines Containers, verschwindet mit ihm.

Secret (Compose)

Als Datei unter `/run/secrets/` bereitgestelltes Geheimnis.

SBOM

Software-Stückliste der im Image enthaltenen Pakete und Komponenten.

Service (Compose)

Ein Dienst in `compose.yaml`, aus dem ein oder mehrere Container entstehen.

Swarm-Modus

In Docker eingebaute Orchestrierung über mehrere Rechner, heute ein Nischenprodukt.

Tag

Beweglicher Name für ein Image, z.B. `1.4.0`.

Volume

Von Docker verwalteter, dauerhafter Datenspeicher.

Quellen und weiterführende Literatur zu den Begriffen und Werkzeugen dieses Handbuchs stehen im anschließenden Literaturverzeichnis. Für Gitea, Pull Requests, Gitea Actions, Runner, Secrets und SSH-Deployment ist das Begleitbuch „Git von Grund auf“ [1] der Einstieg.

Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Online-Quellen wurden am 1. Oktober 2026 abgerufen. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.

- [1] P. Haindl, *Git von Grund auf: Versionsverwaltung am Mac, Pull Requests und CI/CD mit Gitea*. 2026.
- [2] FastAPI, „FastAPI in Containers - Docker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/deployment/docker/>
- [3] Astral Software Inc., „Using uv in Docker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/uv/guides/integration/docker/>
- [4] Docker, Inc., „Docker Engine version 29 release notes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/release-notes/29/>
- [5] Docker, Inc., „Releases · docker/compose“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/docker/compose/releases>
- [6] Docker, Inc., „Deprecated and retired Docker products and features“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/retired/>
- [7] Docker, Inc., „What is an image?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>
- [8] Docker, Inc., „What is Docker?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/get-started/docker-overview/>
- [9] Docker, Inc., „What is a container?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [10] M. Souppaya, J. Morello, und K. Scarfone, „Application Container Security Guide“, National Institute of Standards and Technology, NIST SP 800-190, Sep. 2017. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://csrc.nist.gov/pubs/sp/800/190/final>
- [11] D. Merkel, „Docker: Lightweight Linux Containers for Consistent Development and Deployment“, *Linux Journal*, Mai 2014, Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- [12] Linux man-pages project, „namespaces(7) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- [13] Linux man-pages project, „pid_namespaces(7) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html
- [14] Linux man-pages project, „cgroups(7) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- [15] Docker, Inc., „Storage drivers“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/storage/drivers/>
- [16] Docker, Inc., „Virtual Machine Manager“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/desktop/features/vmm/>
- [17] Docker, Inc., „Docker Engine security“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/security/>
- [18] Docker, Inc., „docker image pull“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/image/pull/>
- [19] Docker, Inc., „Building best practices“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/best-practices/>
- [20] Docker, Inc., „dockerd“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/dockerd/>
- [21] abiosoft, „abiosoft/colima: Container runtimes on macOS (and Linux) with minimal setup“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/abiosoft/colima>
- [22] Docker, Inc., „Docker Desktop license agreement“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/subscription-billing/desktop-license/>
- [23] Orbital Labs, LLC, „Licensing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.orbstack.dev/licensing>
- [24] Podman project, „What is Podman?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.podman.io/en/latest/>
- [25] Homebrew, „Homebrew Formulae: docker-compose“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://formulae.brew.sh/formula/docker-compose>

- [26] Docker, Inc., „docker login“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/login/>
- [27] Abiosoft, „Configuration“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://colima.run/docs/configuration/>
- [28] Apple Inc., „Running Intel Binaries in Linux VMs“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.apple.com/documentation/virtualization/running-intel-binaries-in-linux-vm>
- [29] Abiosoft, „Commands“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://colima.run/docs/commands/>
- [30] abiosoft, „colima/docs/FAQ.md at main“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/abiosoft/colima/blob/main/docs/FAQ.md>
- [31] Abiosoft, „Profiles“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://colima.run/docs/profiles/>
- [32] Docker, Inc., „Docker contexts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/manage-resources/contexts/>
- [33] Docker, Inc., „docker system df“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/system/df/>
- [34] Docker, Inc., „Understand permission requirements for Docker Desktop on Mac“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/desktop/setup/install/mac-permission-requirements/>
- [35] Docker, Inc., „docker container run“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/container/run/>
- [36] Docker, Inc., „Volumes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/storage/volumes/>
- [37] Docker, Inc., „Start containers automatically“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/containers/start-containers-automatically/>
- [38] Docker, Inc., „Prune unused Docker objects“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/manage-resources/pruning/>
- [39] Docker, Inc., „docker container exec“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/container/exec/>
- [40] Docker, Inc., „docker container stop“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/container/stop/>
- [41] Docker, Inc., „docker container inspect“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/container/inspect/>
- [42] GoogleContainerTools, „"Distroless" Container Images“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/GoogleContainerTools/distroless>
- [43] Docker, Inc., „postgres - Official Image“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://hub.docker.com/_/postgres
- [44] Docker, Inc., „docker volume prune“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/volume/prune/>
- [45] Docker, Inc., „docker system prune“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/system/prune/>
- [46] Docker, Inc., „docker compose down“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/down/>
- [47] Docker, Inc., „docker image tag“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/image/tag/>
- [48] Astral Software Inc., „Locking and syncing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/uv/concepts/projects/sync/>
- [49] Docker, Inc., „Dockerfile reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/dockerfile/>
- [50] Docker, Inc., „Build context“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/concepts/context/>
- [51] Docker, Inc., „Docker build cache“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/cache/>
- [52] Docker, Inc., „Optimize cache usage in builds“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/cache/optimize/>
- [53] Docker, Inc., „Bind mounts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/storage/bind-mounts/>
- [54] Docker, Inc., „tmpfs mounts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/storage/tmpfs/>
- [55] Tianon, „Change `PGDATA` in 18+ to `/var/lib/postgresql/MAJOR/docker`“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/docker-library/postgres/pull/1259>
- [56] Tianon, „Remove intentionally-breaking "data" symlink and add better detection“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/docker-library/postgres/pull/1372>

- [57] PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: 18.6. Upgrading a PostgreSQL Cluster“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/upgrading.html>
- [58] PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: pg_dump“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/app-pgdump.html>
- [59] PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: 25.1. SQL Dump“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/backup-dump.html>
- [60] PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: 25.2. File System Level Backup“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/backup-file.html>
- [61] PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: pg_dumpall“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/18/app-pg-dumpall.html>
- [62] Docker, Inc., „Bridge network driver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/drivers/bridge/>
- [63] Docker, Inc., „Networking in Compose“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/networking/>
- [64] Docker, Inc., „Port publishing and mapping“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/port-publishing/>
- [65] FastAPI, „FastAPI CLI“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/fastapi-cli/>
- [66] Uvicorn, „Settings“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://uvicorn.dev/settings/>
- [67] Docker, Inc., „Packet filtering and firewalls“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/packet-filtering-firewalls/>
- [68] Docker, Inc., „Explore networking how-tos on Docker Desktop“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/desktop/features/networking/networking-how-tos/>
- [69] Docker, Inc., „Define services in Docker Compose“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/compose-file/services/>
- [70] abiosoft, „colima/embedded/defaults/colima.yaml at main“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/abiosoft/colima/blob/main/embedded/defaults/colima.yaml>
- [71] Docker, Inc., „Network drivers“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/drivers/>
- [72] Docker, Inc., „Host network driver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/drivers/host/>
- [73] Docker, Inc., „Overlay network driver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/drivers/overlay/>
- [74] nicolaka, „nicolaka/netshoot: a Docker + Kubernetes network trouble-shooting swiss-army container“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nicolaka/netshoot>
- [75] Docker, Inc., „How Compose works“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/intro/compose-application-model/>
- [76] Docker, Inc., „Version and name top-level elements“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/compose-file/version-and-name/>
- [77] Docker, Inc., „Specify a project name“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/project-name/>
- [78] Docker, Inc., „docker compose up“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/up/>
- [79] Docker, Inc., „docker compose config“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/config/>
- [80] Docker, Inc., „Set, use, and manage variables in a Compose file with interpolation“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/>
- [81] Docker, Inc., „Set environment variables within your container's environment“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/environment-variables/set-environment-variables/>
- [82] Docker, Inc., „Release v5.3.0“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/docker/compose/releases/tag/v5.3.0>
- [83] Docker, Inc., „Use init containers in Compose“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/init-containers/>
- [84] Docker, Inc., „Use Compose Watch“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/file-watch/>
- [85] Docker, Inc., „Using profiles with Compose“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/profiles/>
- [86] Docker, Inc., „Merge Compose files“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/>
- [87] Docker, Inc., „python - Official Image | Docker Hub“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://hub.docker.com/_/python

- [88] T.-p. Chung, „PEP 656 – Platform Tag for Linux Distributions Using Musl“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://peps.python.org/pep-0656/>
- [89] Docker, Inc., „Multi-stage builds“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/multi-stage/>
- [90] Astral Software Inc., „Environment variables“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/uv/reference/environment/>
- [91] Docker, Inc., „Build secrets“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/secrets/>
- [92] Renovate, „Docker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/docker/>
- [93] Docker, Inc., „Docker Hardened Images“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/dhi/>
- [94] Open Container Initiative, „Annotations“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/opencontainers/image-spec/blob/main/annotations.md>
- [95] Docker, Inc., „Multi-platform builds“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/multi-platform/>
- [96] Open Container Initiative, „OCI Image Index Specification“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/opencontainers/image-spec/blob/main/image-index.md>
- [97] Distribution project, „Image Manifest Version 2, Schema 2“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/distribution/distribution/blob/main/docs/content/spec/manifest-v2-2.md>
- [98] Docker, Inc., „containerd image store with Docker Engine“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/storage/containerd/>
- [99] Docker, Inc., „Docker container driver“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/builders/drivers/docker-container/>
- [100] Docker, Inc., „docker compose run“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/run/>
- [101] Docker, Inc., „Running containers“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/containers/run/>
- [102] GNU Project, „Exit Status (Bash Reference Manual)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://www.gnu.org/software/bash/manual/html_node/Exit-Status.html
- [103] Linux man-pages project, „signal(7) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man7.org/linux/man-pages/man7/signal.7.html>
- [104] Docker, Inc., „Resource constraints“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://docs.docker.com/engine/containers/resource_constraints/
- [105] Docker, Inc., „docker buildx build“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/buildx/build/>
- [106] Linux man-pages project, „proc_pid_root(5) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://man7.org/linux/man-pages/man5/proc_pid_root.5.html
- [107] Gitea, „Container Registry“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/packages/container>
- [108] Gitea, „API Usage“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/development/api-usage>
- [109] Gitea, „Overview“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/packages/overview>
- [110] Gitea, „Storage“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/packages/storage>
- [111] Docker, Inc., „Docker Hub pull usage and limits“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/docker-hub/usage/pulls/>
- [112] Docker, Inc., „Trusted content“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/docker-hub/image-library/trusted-content/>
- [113] M. Souppaya, J. Morello, und K. Scarfone, „Application Container Security Guide“, SP800–190, Sep. 2017. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>
- [114] Docker, Inc., „Linux post-installation steps for Docker Engine“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/install/linux-postinstall/>
- [115] OWASP Foundation, „Docker Security Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- [116] Docker, Inc., „Rootless mode“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/security/rootless/>
- [117] Docker, Inc., „Troubleshooting“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/security/rootless/troubleshoot/>
- [118] Docker, Inc., „Compose Deploy Specification“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/compose-file/deploy/>

- [119] Docker Official Images (docker-library), „docker-entrypoint.sh“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/docker-library/postgres/blob/master/docker-entrypoint.sh>
- [120] Docker, Inc., „Manage secrets securely in Docker Compose“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/compose/how-tos/use-secrets/>
- [121] Docker, Inc., „mariadb - Official Image“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://hub.docker.com/_/mariadb
- [122] Trivy, „Container Image“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://trivy.dev/docs/latest/target/container_image/
- [123] Trivy, „Others“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://trivy.dev/docs/latest/configuration/others/>
- [124] Sigstore, „Verifying Signatures“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sigstore.dev/cosign/verifying/verify/>
- [125] Notary Project, „Quickstart: Sign and validate a container image“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://notaryproject.dev/docs/quickstart-guides/quickstart-sign-image-artifact/>
- [126] Docker, Inc., „Build attestations“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/metadata/attestations/>
- [127] Docker, Inc., „Seccomp security profiles for Docker“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/security/seccomp/>
- [128] Docker, Inc., „Install Docker Engine on Debian“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/install/debian/>
- [129] Docker, Inc., „Deprecated Docker Engine features“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/deprecated/>
- [130] Docker, Inc., „Configure logging drivers“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/logging/configure/>
- [131] Docker, Inc., „Local file logging driver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/logging/drivers/local/>
- [132] Docker, Inc., „Live restore“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/containers/live-restore/>
- [133] Caddy, „Automatic HTTPS“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://caddyserver.com/docs/automatic-https>
- [134] D. Sato, „Parallel Change“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://martinfowler.com/bliki/ParallelChange.html>
- [135] containrrr, „containrrr/watchtower: A process for automating Docker container base image updates“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/containrrr/watchtower>
- [136] crazy-max, „crazy-max/diun: Receive notifications when an image is updated on a Docker registry“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/crazy-max/diun>
- [137] Docker, Inc., „docker compose exec“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/exec/>
- [138] The PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: pg_dump“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/app-pgdump.html>
- [139] The PostgreSQL Global Development Group, „PostgreSQL: Documentation: 18: pg_restore“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/app-pgrestore.html>
- [140] restic contributors, „Introduction“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://restic.readthedocs.io/en/stable/010_introduction.html
- [141] Synology Inc., „Project | Container Manager - Synology Knowledge Center“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://kb.synology.com/en-us/DSM/help/ContainerManager/docker_project
- [142] GitHub, Inc., „Secure use reference“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/security/secure-use>
- [143] GitHub, Inc., „Using secrets in GitHub Actions“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>
- [144] Gitea, „Frequently Asked Questions“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/faq>
- [145] Gitea, „gitea/runner: Official runner for Gitea“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gitea.com/gitea/runner>
- [146] Docker, Inc., „Provenance attestations“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/metadata/attestations/slsa-provenance/>
- [147] Trivy, „Image“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://trivy.dev/docs/latest/references/configuration/cli/trivy_image/
- [148] Gitea, „Compared to GitHub Actions“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/comparison>
- [149] Sigstore, „Overview“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sigstore.dev/cosign/signing/overview/>

- [150] OpenBSD, „sshd(8) - OpenBSD manual pages“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man.openbsd.org/sshd.8>
- [151] Docker, Inc., „Swarm mode“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/swarm/>
- [152] Docker, Inc., „Docker with nftables“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/network/firewall-nftables/>
- [153] S. Goncharov, „Mirantis Kubernetes Engine 4 Released“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.mirantis.com/blog/mirantis-kubernetes-engine-4-released/>
- [154] K3s Project Authors, „K3s - Lightweight Kubernetes“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.k3s.io/>
- [155] Docker, Inc., „Apply rolling updates to a service“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/swarm/swarm-tutorial/rolling-update/>
- [156] The Kubernetes Authors, „Overview“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://kubernetes.io/docs/concepts/overview/>
- [157] Kamal, „Kamal - Deploy web apps anywhere“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://kamal-deploy.org/>
- [158] Kamal, „Kamal - Proxy“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://kamal-deploy.org/docs/configuration/proxy/>
- [159] Docker, Inc., „docker“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/>